

**PANEVROPSKI UNIVERZITET APEIRON, BANJA LUKA
FAKULTET INFORMACIONIH TEHNOLOGIJA**

Redovne studije

Smjer : “Inženjering Informacionih Tehnologija“

Predmet:

Baze podataka

**ONLINE REZERVACIJA HOTELSKIH SOBA
(Seminarski rad)**

Predmetni nastavnik

Doc.dr Dražen Marinković

Asistent

Milica Radulović

Student: Ivan Pavlović

Br. Indeksa: 92-20 RITP-S

Banja Luka, april 2026

SADRŽAJ

UVOD	1
1. Kreiranje baze podataka.....	2
1.1. Fizička struktura baze podataka.....	2
1.2. Aktiviranje baze podataka i postavljanje kompatibilnosti u SQL Server 2022 (za SQL Server 2014).....	2
2. Kreiranje tabela.....	4
2.1. Tabela “Gosti”	4
2.2. Tabela “Sobe”	5
2.3. Tabela “Rezervacije”	6
2.4. Tabela “Usluge”	7
2.5. Tabela “RezervacijeUsluge”	8
2.6. Tabela “Placanja”	9
3. Uspostavljanje integriteta nad podacima	10
3.1. Integritet na nivou entiteta	10
3.2. Integritet na nivou domena	11
3.3. Ograničenja za tabelu “Gosti”	11
3.4. Ograničenja za tabelu “Sobe”	11
3.5. Ograničenja za tabelu “Rezervacije”	11
3.6. Ograničenja za tabelu “Usluge”	12
3.7. Ograničenja za tabelu “Placanja”	12
3.8. Referencijalni integritet u bazi.....	12
4. Kreiranje indeksa za optimizaciju performansi	14
4.1. Indeks na kolonama „Prezime“ i „Email“ u tabeli „Gosti“	14
4.2. Indeksi na datumskim kolonama u tabeli „Rezervacije“	15
4.3. Indeks na koloni „StatusRezervacije“ u tabeli „Rezervacije“	15
4.4. Kompozitni indeks na kolonama „GostID“ i „StatusRezervacije“	15
4.5. Kompozitni indeks na kolonama „TipSobe“ i „CijenaPoNoci“ u tabeli „Sobe“	15
5. Unos podataka u bazu podataka.....	17
5.1. Unos podataka u tabelu „Gosti“	17
5.2. Unos podataka u tabelu „Sobe“	17
5.3. Unos podataka u tabelu „Rezervacije“	18
5.4. Unos podataka u tabelu „Usluge“	18
5.5. Unos podataka u tabelu „RezervacijeUsluge“	19

5.6.	Unos podataka u tabelu Placanja	19
6.	Kreiranje pogleda (engl. Views) za jednostavniji pristup podacima	20
6.1.	Pogled „vw_RezervacijeDetalji“	20
6.2.	Pogled „vw_PrihodiPoSobama“	21
7.	Kreiranje uskladištenih procedura za automatizaciju	22
7.1.	Procedura „sp_KreirajRezervaciju“	22
7.2.	Procedura „sp_IzracunajUkupnuCijenu“	23
7.3.	Procedura „sp_IzvestajPopunjenosti“	24
8.	Kreiranje trigera za automatsko izvršavanje radnji	25
8.1.	Tabela za praćenje promjena cijena soba.....	25
8.2.	Trigger „trg_OtkaziRezervaciju“	25
8.3.	Trigger „trg_LogCijeneSoba“	26
8.4.	Trigger „trg_ProvjeraDuplihRezervacija“	27
9.	Korištenje korisničkih funkcija za ponovno korištenje koda.....	28
9.1.	Funkcija „fn_IzracunajPopust“	28
9.2.	Funkcija „fn_RaspoloziveSobe“	28
10.	Primjeri DML naredbi za manipulaciju podacima.....	30
10.1.	SELECT naredba za dohvaćanje podataka.....	30
10.2.	INSERT naredba za unos novih podataka	31
10.3.	UPDATE naredba za izmjena postojećih podataka	31
10.4.	DELETE naredba za brisanje podataka	32
11.	Primjer DDL naredbi za definisanje strukture	33
11.1.	ALTER naredba za izmjenu strukture	33
11.2.	DROP naredba za brisanje objekata	33
11.3.	CREATE naredba za kreiranje objekata	33
11.4.	TRUNCATE naredba za brzo brisanje svih podataka	34
12.	Primjeri korištenja različitih tipova funkcija	35
12.1.	Datumske funkcije	35
12.2.	String funkcije.....	35
12.3.	Agregatne funkcije.....	36
13.	Primjeri JOIN upita za kombinovanje podataka iz više tabela	37
13.1.	INNER JOIN.....	37
13.2.	LEFT JOIN (LEFT OUTER JOIN)	37
13.3.	RIGHT JOIN (RIGHT OUTER JOIN).....	38

13.4.	FULL OUTER JOIN.....	38
13.5.	CROSS JOIN	39
13.6.	SELF JOIN.....	39
14.	Kreiranje rezervne kopije baze podataka.....	40
14.1.	Kreiranje potpunog backup-a (Full Backup)	40
14.2.	Druge vrste backup-a	40
14.2.1.	Diferencijalni backup (Differential Backup)	41
14.2.2.	Backup loga transakcija (Transaction Log Backup).....	41
14.2.3.	Copy-only backup.....	41
14.3.	Restauracija baze iz backup-a.....	41
15.	ER dijagram i opis relacija.....	43
15.1.	Entiteti i njihovi atributi.....	43
15.2.	Veze između entiteta.....	43
15.3.	Grafički prikaz ER dijagrama	44
ZAKLJUČAK.....		45
POPIS SLIKA.....		46
RESURSI I DODATNI MATERIJALI		48

UVOD

Savremeno turističko tržište karakteriše visok nivo konkurencije i stalno rastuća očekivanja korisnika usluga. U takvom okruženju, hotelska industrija suočava se sa potrebom za efikasnim upravljanjem kapacitetima, rezervacijama i odnosima sa gostima. Tradicionalni načini vođenja evidencije, poput papirne dokumentacije ili jednostavnih tabelarnih prikaza, više ne mogu odgovoriti na zahtjeve brzine, tačnosti i pouzdanosti koji se danas podrazumijevaju.

Digitalizacija poslovnih procesa postaje neizostavan dio modernog hotelijerstva. Posebno značajnu ulogu imaju informacioni sistemi koji omogućavaju automatizaciju ključnih aktivnosti, smanjenje mogućnosti grešaka i unapređenje korisničkog iskustva. Sistemi za online rezervaciju predstavljaju važan segment takvog pristupa, jer omogućavaju jednostavno upravljanje dostupnošću smještajnih kapaciteta, pregled rezervacija u realnom vremenu i efikasnu komunikaciju sa gostima.

Projektovanje i implementacija baze podataka predstavljaju osnovu svakog pouzdanog informacionog sistema. Kvalitetno dizajniran relacioni model omogućava dosljedno čuvanje podataka, njihovu povezanost i jednostavnu obradu. U kontekstu hotelskog poslovanja, takav model obuhvata evidenciju gostiju, soba, rezervacija, dodatnih usluga i finansijskih transakcija, uz jasno definisane veze između tih elemenata.

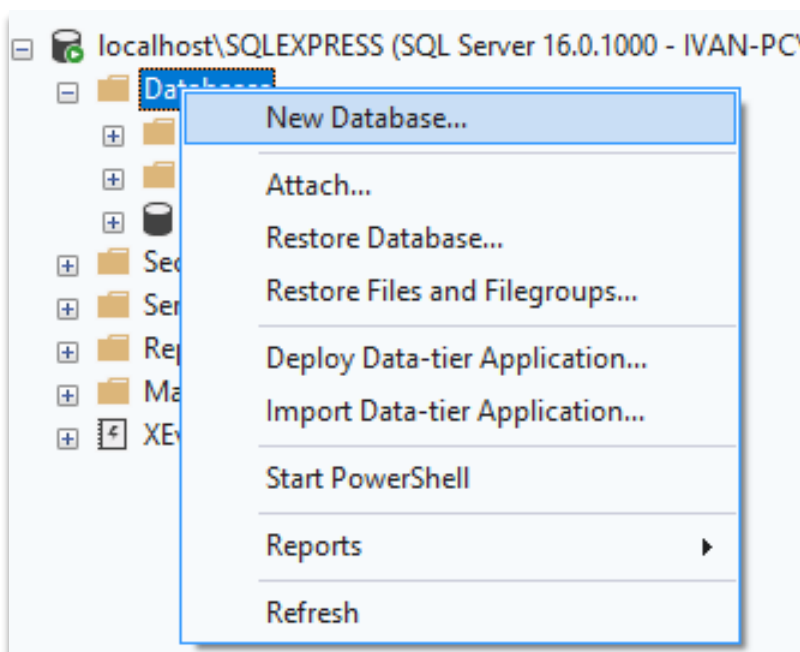
Za realizaciju baze podataka korišten je Microsoft SQL Server 2022, uz odgovarajući nivo kompatibilnosti koji obezbjeđuje stabilan i pouzdan rad sistema. Posebna pažnja posvećena je očuvanju integriteta podataka, kao i optimizaciji performansi kroz pravilno definisane indekse. Pored osnovnih tabela, korišćeni su i napredni objekti baze podataka, uključujući poglede, uskladištene procedure, trigere i korisnički definisane funkcije, čime se omogućava efikasnija obrada podataka i automatizacija poslovnih procesa.

Na ovaj način formiran je cjelovit i funkcionalan model baze podataka koji odgovara savremenim zahtjevima hotelskog poslovanja i predstavlja pouzdanu osnovu za dalji razvoj informacionog sistema.

1. Kreiranje baze podataka

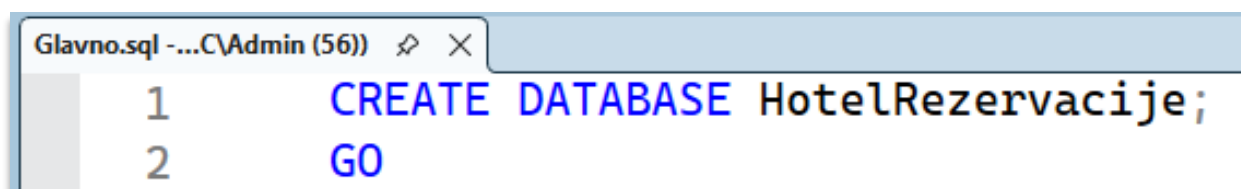
1.1. Fizička struktura baze podataka

Prvi korak u izradi projekta jeste kreiranje baze podataka pod nazivom „HotelRezervacije“. Time se uspostavlja osnovna struktura sistema u kojoj će se čuvati i organizovati svi podaci vezani za hotelsko poslovanje.



Slika 1 - Kreiranje baze podataka pomoću GUI-a

U okviru sistema SQL Server, baza podataka predstavlja logičku cjelinu koja objedinjuje sve tabele, veze i ostale objekte potrebne za rad aplikacije. Njeno kreiranje predstavlja početnu tačku u razvoju svakog informacionog sistema zasnovanog na bazi podataka.



Slika 2 - Kreiranje baze podataka pomoću Query-a

1.2. Aktiviranje baze podataka i postavljanje kompatibilnosti u SQL Server 2022 (za SQL Server 2014)

Nakon kreiranja baze podataka, potrebno je postaviti aktivnu bazu za dalji rad. To se postiže korištenjem naredbe **USE**, kojom se definiše da će se sve naredne operacije izvršavati nad bazom „HotelRezervacije“.

```
4      USE HotelRezervacije;  
5      GO
```

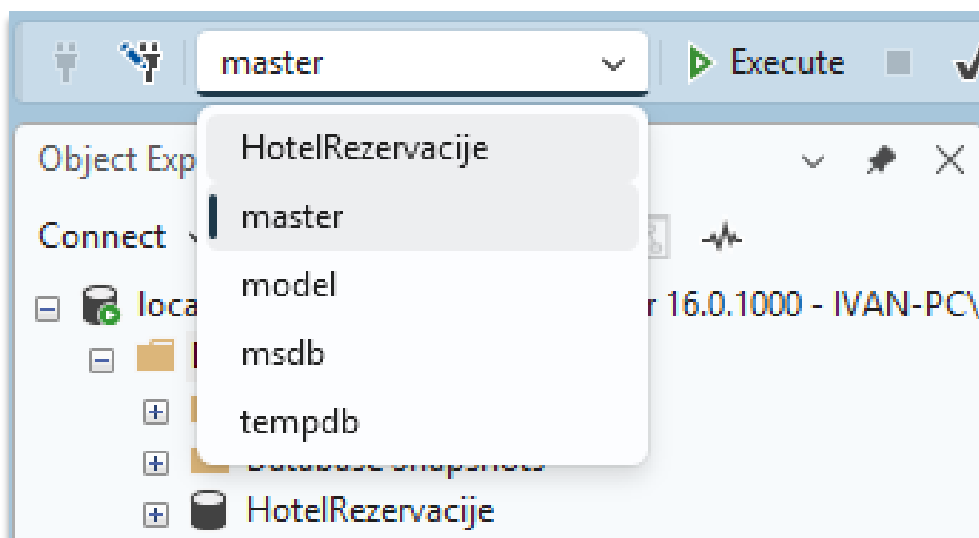
Slika 3 - Aktiviranje baze podataka

Pored toga, definiše se i nivo kompatibilnosti baze podataka. Ovim podešavanjem određuje se način na koji SQL Server izvršava SQL upite u skladu sa određenom verzijom sistema. Za SQL Server 2014 koristi se nivo kompatibilnosti 120, čime se obezbeđuje stabilan i očekivan rad baze podataka.

```
7      ALTER DATABASE HotelRezervacije  
8      SET COMPATIBILITY_LEVEL = 120;  
9      GO
```

Slika 4 - Postavljanje nivoa kompatibilnosti baze podataka na SQL Server 2014 (nivo 120)

Također, aktivna baza se može izabrati i putem grafičkog korisničkog interfejsa u okviru SQL Server Management Studio okruženja, gdje se u gornjem lijevom dijelu prozora nalazi padajući meni za izbor baze podataka. Ovaj način je često jednostavniji za korištenje prilikom rada u okruženju sa više baza.



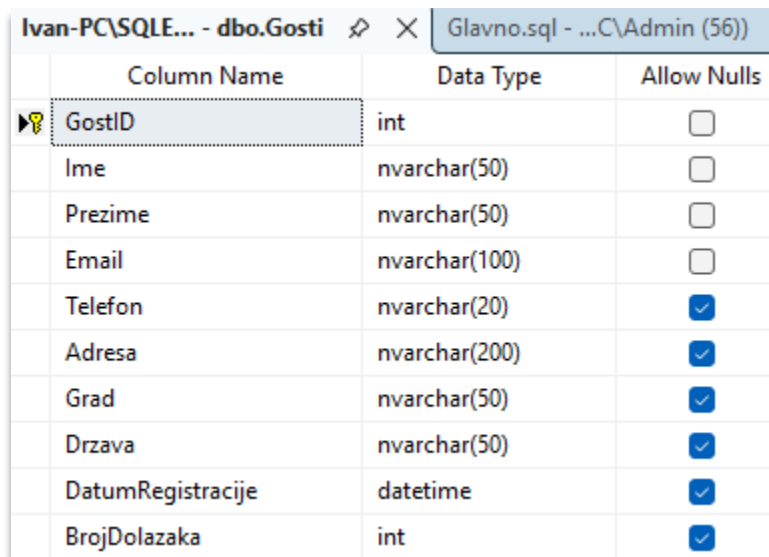
Slika 5 - Selektovanje aktive baze putem GUI-a

2. Kreiranje tabela

Nakon kreiranja baze podataka, slijedi definisanje tabela koje će čuvati podatke o glavnim entitetima sistema. Svaka tabela sadrži kolone sa odgovarajućim tipovima podataka, a za svaku je definisan primarni ključ radi jedinstvene identifikacije zapisa. Veze između tabela ostvarene su pomoću stranih ključeva.

2.1. Tabela "Gosti"

Tabela Gosti predstavlja centralni entitet u sistemu jer čuva sve informacije o osobama koje koriste hotelske usluge. Ova tabela je ključna za identifikaciju gostiju, praćenje njihove istorije boravka i uspostavljanje dugoročnih odnosa sa stalnim gostima.



Column Name	Data Type	Allow Nulls
GostID	int	☐
Ime	nvarchar(50)	☐
Prezime	nvarchar(50)	☐
Email	nvarchar(100)	☐
Telefon	nvarchar(20)	☒
Adresa	nvarchar(200)	☒
Grad	nvarchar(50)	☒
Drzava	nvarchar(50)	☒
DatumRegistracije	datetime	☒
BrojDolazaka	int	☒

Slika 6 - Prikaz tabele "Gosti"

Prva kolona u ovoj tabeli je GostID, koja je definisana kao IDENTITY kolona. Identity svojstvo znači da će SQL Server automatski generisati jedinstvenu numeričku vrijednost za svaki novi red, počevši od 1 i povećavajući se za 1. Ova kolona je postavljena kao primarni ključ tabele, što znači da će svaki gost imati svoj jedinstveni identifikator koji se koristi za povezivanje sa drugim tabelama.

GostID **INT IDENTITY(1,1) PRIMARY KEY**

Kolona Ime i Prezime definisane su kao NVARCHAR(50) sa ograničenjem NOT NULL, što znači da ova polja ne mogu ostati prazna prilikom unosa novog gosta. NVARCHAR tip podataka omogućava čuvanje Unicode znakova, što je važno za podršku različitih jezika i specijalnih karaktera.

Ime **NVARCHAR(50) NOT NULL**,
Prezime **NVARCHAR(50) NOT NULL**

Email adresa je definisana kao NVARCHAR(100) sa dva važna ograničenja, NOT NULL i UNIQUE. Ograničenje NOT NULL osigurava da svaki gost mora imati email adresu, dok UNIQUE ograničenje garantuje da se ista email adresa ne može pojaviti više puta, što je važno za autentifikaciju korisnika u online sistemu. Telefon, Adresa, Grad i Država su opcionalna polja koja mogu biti NULL, što znači da gost nije u obavezi da ih popuni prilikom registracije. DatumRegistracije je definisan sa DEFAULT GETDATE(), što znači da će se prilikom unosa novog gosta automatski upisati trenutni datum i vrijeme registracije, bez potrebe da korisnik ili aplikacija to ručno unose. BrojDolazaka je numeričko polje sa podrazumijevanom vrijednošću 0. Ovo polje će se automatski povećavati svaki put kada gost izvrši novu rezervaciju, što omogućava praćenje lojalnosti gostiju i potencijalno dodjeljivanje popusta stalnim gostima.

```
11      CREATE TABLE Gosti (
12          GostID INT IDENTITY(1,1) PRIMARY KEY,
13          Ime NVARCHAR(50) NOT NULL,
14          Prezime NVARCHAR(50) NOT NULL,
15          Email NVARCHAR(100) NOT NULL UNIQUE,
16          Telefon NVARCHAR(20),
17          Adresa NVARCHAR(200),
18          Grad NVARCHAR(50),
19          Drzava NVARCHAR(50),
20          DatumRegistracije DATETIME DEFAULT GETDATE(),
21          BrojDolazaka INT DEFAULT 0,
22          CONSTRAINT CHK_Email CHECK (Email LIKE '%_@_%._%')
23      );
```

Slika 7 - Query korišten za kreiranje tabele "Gosti"

2.2. Tabela "Sobe"

Tabela Sobe predstavlja inventar hotelskih kapaciteta. Svaka soba u hotelu ima svoj zapis u ovoj tabeli, sa svim relevantnim informacijama koje su potrebne za proces rezervacije i upravljanja kapacitetima.

SobaID je primarni ključ tabele, definisan kao IDENTITY kolona koja automatski generiše jedinstveni identifikator za svaku sobu.

SobaID INT IDENTITY(1,1) PRIMARY KEY

BrojSobe je kolona koja čuva stvarni broj sobe kako je označen u hotelu. Ovo polje ima ograničenje UNIQUE, što znači da ne mogu postojati dvije sobe sa istim brojem. Također je postavljeno kao NOT NULL jer svaka soba mora imati svoj identifikacioni broj.

BrojSobe NVARCHAR(10) NOT NULL UNIQUE

TipSobe definiše vrstu sobe, kao što su Single, Double ili Suite. Ovo polje je važno za pretragu i filtriranje soba prilikom online rezervacije. Kapacitet sobe definiše maksimalan broj osoba koje mogu boraviti u sobi. Ovo polje je postavljeno kao NOT NULL jer svaka soba mora imati definisan kapacitet.

CijenaPoNoci je decimalna kolona koja čuva cijenu boravka po noći. Decimalni tip podataka je odabran jer se radi o novčanim iznosima gdje je potrebna preciznost do dvije decimale.

CijenaPoNoci **DECIMAL(10,2) NOT NULL**

Sprat i Pogled su opcionalna polja koja pružaju dodatne informacije o lokaciji i karakteristikama sobe. StatusSobe je posebno važno polje za online sistem rezervacija. Ono prati trenutno stanje sobe i može imati vrijednosti, Dostupna, Zauzeta, Rezervisana ili Na čiscenju. Podrazumijevana vrijednost je Dostupna, što znači da su sve sobe inicijalno dostupne za rezervaciju.

StatusSobe **NVARCHAR(20) DEFAULT 'Dostupna'**

```
CREATE TABLE Sobe (  
    SobaID INT IDENTITY(1,1) PRIMARY KEY,  
    BrojSobe NVARCHAR(10) NOT NULL UNIQUE,  
    TipSobe NVARCHAR(50) NOT NULL,  
    Kapacitet INT NOT NULL,  
    CijenaPoNoci DECIMAL(10,2) NOT NULL,  
    Sprat INT,  
    Pogled NVARCHAR(50),  
    StatusSobe NVARCHAR(20) DEFAULT 'Dostupna',  
    CONSTRAINT CHK_Cijena CHECK (CijenaPoNoci > 0),  
    CONSTRAINT CHK_Kapacitet CHECK (Kapacitet BETWEEN 1 AND 10),  
    CONSTRAINT CHK_Status CHECK (StatusSobe IN ('Dostupna', 'Zauzeta', 'Rezervisana', 'Na čiscenju'))  
);
```

Slika 8 - Query korišten za kreiranje tabele "Sobe"

2.3. Tabela "Rezervacije"

Tabela Rezervacije je srce cjelokupnog sistema jer povezuje goste sa sobama i čuva sve informacije o samom procesu rezervacije. Ova tabela sadrži podatke o tome ko je gost, koju sobu je rezervisao, u kojem periodu, kao i dodatne informacije koje su važne za pravilno izvršavanje usluge.

RezervacijaID je primarni ključ tabele, definisan kao IDENTITY kolona koja automatski generiše jedinstveni identifikator za svaku rezervaciju. GostID i SobaID su strani ključevi koji povezuju rezervaciju sa konkretnim gostom i konkretnom sobom. Ove kolone su postavljene kao NOT NULL jer svaka rezervacija mora biti vezana za postojećeg gosta i postojeću sobu.

RezervacijaID **INT IDENTITY(1,1) PRIMARY KEY**,
GostID **INT NOT NULL**,
SobaID **INT NOT NULL**,

DatumPrijave i DatumOdjave su kolone tipa DATE koje definišu period boravka. DATE tip je odabran jer nas zanima samo datum, bez vremenske komponente. DatumOdjave mora biti veći od DatumaPrijave, što će biti osigurano kroz ograničenje integriteta. DatumRezervacije je kolona sa podrazumijevanom vrijednošću GETDATE(), što znači da se prilikom kreiranja rezervacije automatski bilježi trenutak kada je rezervacija napravljena. BrojOdraslih i BrojDjece prate strukturu gostiju koji će boraviti u sobi. Ovo je važno za pravilno raspoređivanje kapaciteta i za eventualne dodatne naplate. StatusRezervacije prati trenutno stanje rezervacije i može imati vrijednosti kao što su Potvrđena, Na čekanju ili Otkazana. Podrazumijevana vrijednost je Potvrđena. Napomena je tekstualno polje koje omogućava unos dodatnih informacija ili specijalnih zahtjeva gosta.

```
CREATE TABLE Rezervacije (  
    RezervacijaID INT IDENTITY(1,1) PRIMARY KEY,  
    GostID INT NOT NULL,  
    SobaID INT NOT NULL,  
    DatumPrijave DATE NOT NULL,  
    DatumOdjave DATE NOT NULL,  
    DatumRezervacije DATETIME DEFAULT GETDATE(),  
    BrojOdraslih INT DEFAULT 1,  
    BrojDjece INT DEFAULT 0,  
    StatusRezervacije NVARCHAR(20) DEFAULT 'Potvrđena',  
    Napomena NVARCHAR(500),  
    CONSTRAINT CHK_Datum CHECK (DatumOdjave > DatumPrijave),  
    CONSTRAINT CHK_BrojGostiju CHECK (BrojOdraslih + BrojDjece <= 10),  
    CONSTRAINT FK_Rezervacije_Gosti FOREIGN KEY (GostID) REFERENCES Gosti(GostID),  
    CONSTRAINT FK_Rezervacije_Sobe FOREIGN KEY (SobaID) REFERENCES Sobe(SobaID)  
);
```

Slika 9 - Query korišten za kreiranje tabele "Rezervacije"

2.4. Tabela "Usluge"

Tabela Usluge predstavlja katalog dodatnih usluga koje hotel nudi svojim gostima. Ove usluge se mogu dodavati uz rezervaciju i naplaćivati posebno. Primjeri ovakvih usluga uključuju doručak, večeru, parking, wellness, room service i slično.

UslugaID je primarni ključ tabele, definisan kao IDENTITY kolona koja automatski generiše jedinstveni identifikator za svaku uslugu. NazivUsluge je kolona koja čuva kratak naziv usluge, kao što je "Doručak" ili "Parking". Ovo polje je obavezno (NOT NULL) jer svaka usluga mora imati svoj naziv.

Opis je tekstualno polje koje omogućava detaljniji opis usluge, što može biti korisno za goste koji žele više informacija prije nego što odaberu uslugu. Cijena je decimalna kolona koja čuva cijenu usluge.

Kao i kod cijene soba, decimalni tip podataka osigurava preciznost do dvije decimale. Kategorija omogućava grupisanje usluga po tipovima, kao što su Hrana, Spa, Parking, što olakšava pretragu i prikaz na korisničkom interfejsu.

```
CREATE TABLE Usluge (
    UslugaID INT IDENTITY(1,1) PRIMARY KEY,
    NazivUsluge NVARCHAR(100) NOT NULL,
    Opis NVARCHAR(500),
    Cijena DECIMAL(10,2) NOT NULL,
    Kategorija NVARCHAR(50),
    CONSTRAINT CHK_CijenaUsluge CHECK (Cijena >= 0)
);
```

Slika 10 - Query korišten za kreiranje tabele "Usluge"

2.5. Tabela "RezervacijeUsluge"

Tabela RezervacijeUsluge predstavlja rješenje za vezu više-prema-više između rezervacija i usluga. U realnom svijetu, jedna rezervacija može uključivati više različitih usluga, a jedna usluga može biti dodijeljena više različitih rezervacija. Ovakav odnos se ne može direktno predstaviti u relacionoj bazi podataka bez korištenja pomoćne tabele.

RezervacijaUslugaID je primarni ključ ove pomoćne tabele, koji jedinstveno identifikuje svaku kombinaciju rezervacije i usluge.

RezervacijaUslugaID INT IDENTITY(1,1) PRIMARY KEY

RezervacijaID i UslugaID su strani ključevi koji povezuju ovu tabelu sa tabelama Rezervacije i Usluge. Ove kolone su postavljene kao NOT NULL jer svaki zapis mora biti vezan za konkretnu rezervaciju i konkretnu uslugu.

RezervacijaID INT NOT NULL,
UslugaID INT NOT NULL,

Količina omogućava evidentiranje višestrukih jedinica iste usluge. Na primjer, ako gost želi doručak za svih pet dana boravka, količina će biti 5.

CijenaPoJedinici čuva cijenu usluge u trenutku rezervacije. Ovo je važno jer se cijene usluga mogu mijenjati tokom vremena, a želimo da zabilježimo cijenu koja je važila kada je gost napravio rezervaciju.

```
CREATE TABLE RezervacijeUsluge (
    RezervacijaUslugaID INT IDENTITY(1,1) PRIMARY KEY,
    RezervacijaID INT NOT NULL,
    UslugaID INT NOT NULL,
    Kolicina INT DEFAULT 1,
    CijenaPoJedinici DECIMAL(10,2) NOT NULL,
    CONSTRAINT FK_RezUsl_Rezervacije FOREIGN KEY (RezervacijaID) REFERENCES Rezervacije(RezervacijaID),
    CONSTRAINT FK_RezUsl_Usluge FOREIGN KEY (UslugaID) REFERENCES Usluge(UslugaID)
);
```

Slika 11 - Query korišten za kreiranje tabele "RezervacijeUsluge"

2.6. Tabela “Placanja”

Tabela Placanja služi za evidentiranje svih uplata koje su izvršene za rezervacije. U hotelijerskom poslovanju, jedna rezervacija može imati više uplata, na primjer avans prilikom rezervacije i ostatak po dolasku, što opravdava postojanje tabele Placanja kao zasebnog entiteta.

PlacanjeID je primarni ključ tabele, definisan kao IDENTITY kolona koja automatski generiše jedinstveni identifikator za svaku uplatu. RezervacijaID je strani ključ koji povezuje uplatu sa konkretnom rezervacijom. Ova kolona je postavljena kao NOT NULL jer svaka uplata mora biti vezana za postojeću rezervaciju.

PlacanjeID **INT IDENTITY(1,1) PRIMARY KEY**,
RezervacijaID **INT NOT NULL**,

Iznos je decimalna kolona koja čuva iznos uplate. Ograničenje da iznos mora biti veći od nule osigurava da se ne mogu unijeti negativne ili nulte uplate. DatumPlacanja je kolona sa podrazumijevanom vrijednošću GETDATE(), što znači da se prilikom unosa uplate automatski bilježi trenutak kada je uplata izvršena.

Iznos **DECIMAL(10,2) NOT NULL**,
DatumPlacanja **DATETIME DEFAULT GETDATE()**,

NacinPlacanja definiše način na koji je uplata izvršena, a može imati vrijednosti Gotovina, Kartica ili PayPal. Ovo je važno za finansijsko izvještavanje i praćenje različitih kanala naplate. StatusPlacanja prati trenutno stanje uplate, sa mogućim vrijednostima “Na čekanju” ili “Potvrđeno”. TransakcijaID je polje koje čuva identifikator transakcije iz eksternog sistema za online plaćanja. Ovo je ključno za praćenje i eventualnu povratnu potvrdu online uplata.

NacinPlacanja **NVARCHAR(50) NOT NULL**,
StatusPlacanja **NVARCHAR(20) DEFAULT 'Na čekanju'**,
TransakcijaID **NVARCHAR(100)**,

```
CREATE TABLE Placanja (  
    PlacanjeID INT IDENTITY(1,1) PRIMARY KEY,  
    RezervacijaID INT NOT NULL,  
    Iznos DECIMAL(10,2) NOT NULL,  
    DatumPlacanja DATETIME DEFAULT GETDATE(),  
    NacinPlacanja NVARCHAR(50) NOT NULL,  
    StatusPlacanja NVARCHAR(20) DEFAULT 'Na čekanju',  
    TransakcijaID NVARCHAR(100),  
    CONSTRAINT FK_Placanja_Rezervacije FOREIGN KEY (RezervacijaID) REFERENCES Rezervacije(RezervacijaID),  
    CONSTRAINT CHK_Iznos CHECK (Iznos > 0),  
    CONSTRAINT CHK_NacinPlacanja CHECK (NacinPlacanja IN ('Gotovina', 'Kartica', 'PayPal'))  
);
```

Slika 12 - Query korišten za kreiranje tabele "Placanja"

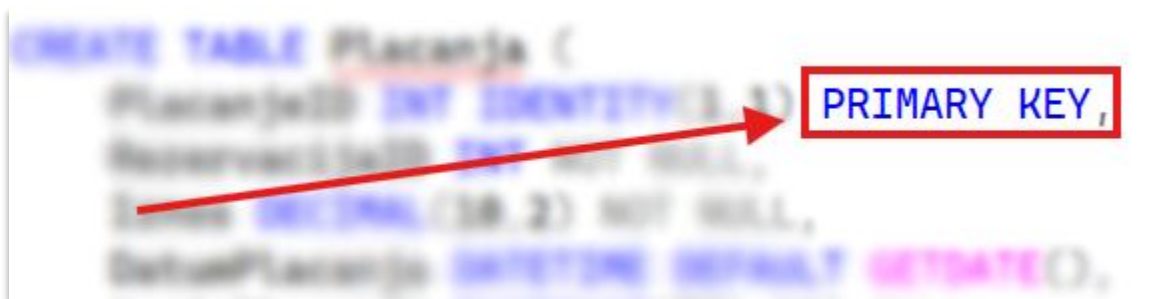
3. Uspostavljanje integriteta nad podacima

Integritet podataka predstavlja jedan od najvažnijih koncepata u projektovanju baza podataka. On osigurava da su podaci u bazi tačni, konzistentni i da zadovoljavaju sve poslovne zahtjeve. U ovoj bazi podataka uspostavljena su tri nivoa integriteta: integritet entiteta, integritet domena i referencijalni integritet.

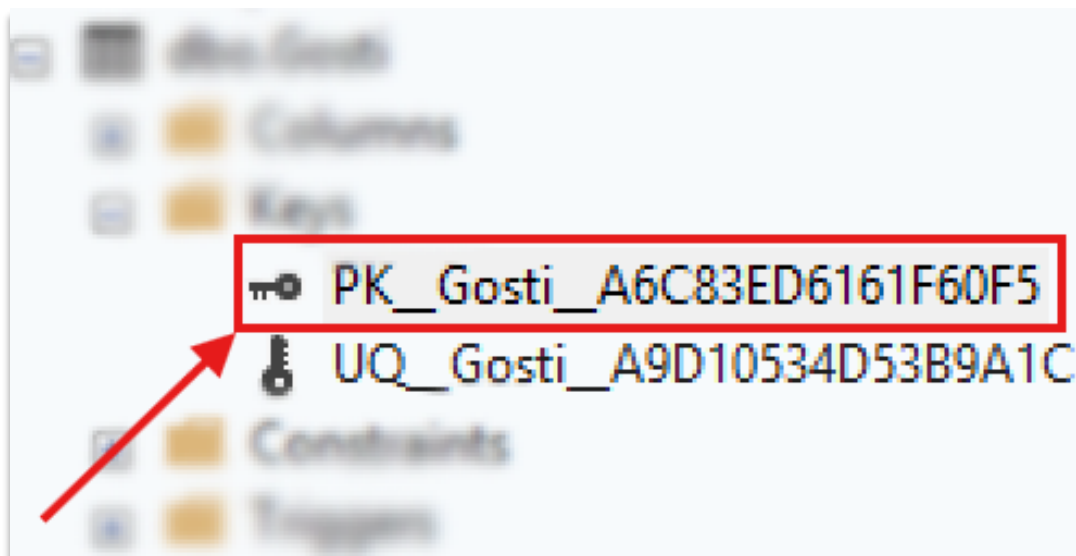
3.1. Integritet na nivou entiteta

Integritet entiteta osigurava da svaki zapis u tabeli bude jedinstven i jasno identifikovan, čime se sprečava pojava duplih podataka koji bi mogli predstavljati isti entitet. U bazi podataka "HotelRezervacije", integritet entiteta postiže se definisanjem primarnih ključeva za sve glavne tabele. Primarni ključevi su postavljeni na kolone GostID, SobaID, RezervacijaID, UslugaID, RezervacijaUslugaID i PlaćanjeID. Ove kolone koriste IDENTITY svojstvo, što omogućava automatsko generisanje jedinstvenih vrijednosti pri unosu novih zapisa, čime se dodatno smanjuje mogućnost greške i osigurava dosljednost podataka u sistemu.

Primarni ključevi predstavljaju osnovu za povezivanje tabela i garantuju da svaki entitet može biti jednoznačno identifikovan, što je od suštinske važnosti za pouzdanost i stabilnost baze podataka.



Slika 13 - Ključna riječ za podešavanje primarnog ključa



Slika 14 - Dodatni način pronalaženja primarnog ključa tabele pomoću GUI-a

3.2. Integritet na nivou domena

Integritet domena se odnosi na ograničenja koja definišu dozvoljene vrijednosti za pojedine kolone. Ovaj oblik integriteta osigurava da podaci unutar baze budu validni i konzistentni, u skladu sa pravilima poslovnog sistema. U praksi se postiže korišćenjem CHECK ograničenja, koja postavljaju konkretna pravila za unos podataka.

```
CONSTRAINT CHK_Iznos CHECK (Iznos > 0),
```

Slika 15 - Primjer korištenja CHECK naredbe

Primjenom integriteta na nivou domena smanjuje se rizik od pogrešnog unosa podataka i obezbjeđuje se visok nivo kvaliteta i pouzdanosti informacija unutar sistema.

3.3. Ograničenja za tabelu “Gosti”

Za tabelu Gosti, dodato je CHECK ograničenje koje provjerava format email adrese. Ograničenje koristi LIKE operator sa šablonom koji osigurava da email sadrži simbol @ i tačku nakon njega, čime se sprječava unos očigledno neispravnih email adresa.

```
CONSTRAINT CHK_Email CHECK (Email LIKE '%_@__%.__%')
```

3.4. Ograničenja za tabelu “Sobe”

Za tabelu Sobe, uspostavljena su tri CHECK ograničenja. Prvo ograničenje provjerava da je cijena po noći veća od nule, čime se sprječava unos besplatnih ili negativnih cijena. Drugo ograničenje osigurava da kapacitet sobe bude između 1 i 10 osoba, što je realan raspon za hotelske sobe. Treće ograničenje definiše dozvoljene vrijednosti za status sobe, što su isključivo, Dostupna, Zauzeta, Rezervisana i Na čišćenju.

```
CONSTRAINT CHK_Cijena CHECK (CijenaPoNoci > 0),  
CONSTRAINT CHK_Kapacitet CHECK (Kapacitet BETWEEN 1 AND 10),  
CONSTRAINT CHK_Status CHECK (StatusSobe IN ('Dostupna', 'Zauzeta', 'Rezervisana',  
'Na ciscenju'))
```

3.5. Ograničenja za tabelu “Rezervacije”

Za tabelu Rezervacije, uspostavljena su dva CHECK ograničenja. Prvo ograničenje provjerava da je datum odjave veći od datuma prijave, što je logički neophodno za ispravnu rezervaciju. Drugo ograničenje osigurava da ukupan broj gostiju (odrasli plus djeca) ne prelazi 10, što je u skladu sa kapacitetom soba.

```
CONSTRAINT CHK_Datum CHECK (DatumOdjave > DatumPrijave),  
CONSTRAINT CHK_BrojGostiju CHECK (BrojOdraslih + BrojDece <= 10)
```


3.6. Ograničenja za tabelu “Usluge”

Za tabelu Usluge, dodato je ograničenje koje provjerava da cijena usluge nije negativna.

```
CONSTRAINT CHK_CijenaUsluge CHECK (Cijena >= 0)
```

3.7. Ograničenja za tabelu “Plaćanja”

Za tabelu Plaćanja, uspostavljena su dva CHECK ograničenja. Prvo ograničenje provjerava da je iznos uplate veći od nule. Drugo ograničenje definiše dozvoljene načine plaćanja kao što su gotovina, kartica i PayPal.

```
CONSTRAINT CHK_Iznos CHECK (Iznos > 0),  
CONSTRAINT CHK_NacinPlacanja CHECK (NacinPlacanja IN ('Gotovina', 'Kartica',  
'PayPal'))
```

3.8. Referencijalni integritet u bazi

Referencijalni integritet osigurava da veze između tabela budu konzistentne. U bazi HotelRezervacije, referencijalni integritet je uspostavljen kroz pet stranih ključeva.

Tabela „Rezervacije” sadrži dva strana ključa. Prvi je GostID koji referencira tabelu Gosti. Ovo znači da se rezervacija može kreirati samo za postojećeg gosta. Drugi je SobaID koji referencira tabelu Sobe, što znači da se rezervacija može kreirati samo za postojeću sobu.

-- Primjer: Ne može se kreirati rezervacija za nepostojećeg gosta

-- Ova naredba će generisati grešku jer gost sa ID 99 ne postoji

```
INSERT INTO Rezervacije (GostID, SobaID, DatumPrijave, DatumOdjave)  
VALUES (99, 1, '2025-05-01', '2025-05-05');
```

-- Rezultat: The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Rezervacije_Gosti"

Tabela “RezervacijeUsluge” sadrži dva strana ključa. RezervacijaID referencuje tabelu Rezervacije, a UslugaID referencira tabelu Usluge. Ovo osigurava da se usluga može dodati samo postojećoj rezervaciji i da se može dodati samo postojeća usluga.

-- Primjer: Ne može se dodati usluga nepostojećoj rezervaciji

```
INSERT INTO RezervacijeUsluge (RezervacijaID, UslugaID, Kolicina, CijenaPoJedinici)  
VALUES (99, 1, 2, 15.00);
```

-- Rezultat: The INSERT statement conflicted with the FOREIGN KEY constraint "FK_RezUsl_Rezervacije"

Tabela "Placanja" sadrži strani ključ RezervacijaID koji referencira tabelu Rezervacije. Ovo osigurava da se plaćanje može evidentirati samo za postojeću rezervaciju.

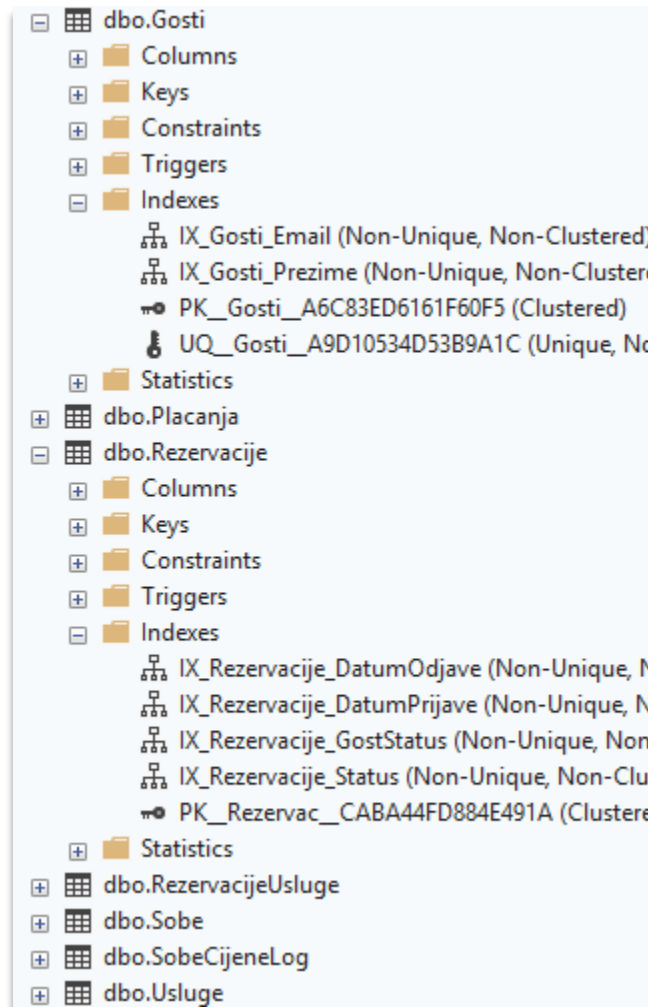
-- Primjer: Ne može se evidentirati plaćanje za nepostojeću rezervaciju

```
INSERT INTO Placanja (RezervacijaID, Iznos, NacinPlacanja)  
VALUES (99, 200.00, 'Kartica');
```

-- Rezultat: The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Placanja_Rezervacije"

4. Kreiranje indeksa za optimizaciju performansi

Indeksi su posebne strukture u bazi podataka koje značajno ubrzavaju izvršavanje upita, posebno onih koji pretražuju, filtriraju ili sortiraju velike količine podataka. Razlog zašto indeksi ubrzavaju pretragu sličan je razlogu zašto indeks na kraju knjige pomaže da brzo pronađemo željeno poglavlje umjesto da listamo cijelu knjigu, pogledamo indeks i direktno odemo na traženu stranicu.



Slika 16 - Prikaz nekih od indeksa u tabelama

4.1. Indeks na kolonama „Prezime“ i „Email“ u tabeli „Gosti“

Prvi indeks je kreiran na koloni Prezime u tabeli Gosti. Ovaj indeks je izuzetno važan jer će se gosti često pretraživati po prezimenu prilikom prijave u hotel ili prilikom kreiranja nove rezervacije. Bez indeksa, SQL Server bi morao pregledati svaki red u tabeli da bi pronašao gosta sa traženim prezimenom, što bi bilo sporo kada tabela naraste na hiljade ili desetine hiljada gostiju.

Drugi indeks je kreiran na koloni Email u tabeli Gosti. Email adresa se koristi za autentifikaciju prilikom prijave u online sistem, tako da je brzo pronalaženje gosta po emailu ključno za dobro korisničko iskustvo.

```
CREATE INDEX IX_Gosti_Prezime ON Gosti(Prezime);  
CREATE INDEX IX_Gosti_Email ON Gosti(Email);
```

Slika 17 - Postavljanje indeksa na kolone Prezime i Email u tabeli "Gosti"

4.2. Indeksi na datumskim kolonama u tabeli „Rezervacije“

Treći i četvrti indeks su kreirani na kolonama DatumPrijave i DatumOdjave u tabeli Rezervacije. Ovi indeksi su važni jer će sistem često izvršavati upite koji pronalaze rezervacije za određeni period, kao što je prikaz svih aktivnih rezervacija za tekući dan ili provjera dostupnosti soba za traženi period. Indeksi na datumskim kolonama omogućavaju da se ovi upiti izvršavaju trenutno, čak i kada tabela sadrži milione zapisa.

```
CREATE INDEX IX_Rezervacije_DatumPrijave ON Rezervacije(DatumPrijave);  
CREATE INDEX IX_Rezervacije_DatumOdjave ON Rezervacije(DatumOdjave);
```

Slika 18 - Postavljanje indeksa na datumske kolone u tabeli "Rezervacije"

4.3. Indeks na koloni „StatusRezervacije“ u tabeli „Rezervacije“

Peti indeks je kreiran na koloni StatusRezervacije u tabeli Rezervacije. Ovaj indeks je važan za upite koji filtriraju rezervacije po statusu, kao što je prikaz svih potvrđenih rezervacija za određeni period ili prikaz otkazanih rezervacija za potrebe analize.

```
CREATE INDEX IX_Rezervacije_Status ON Rezervacije(StatusRezervacije);
```

Slika 19 - Postavljanje indeksa na kolonu StatusRezervacije u tabeli "Rezervacije"

4.4. Kompozitni indeks na kolonama „GostID“ i „StatusRezervacije“

Šesti indeks je kompozitni indeks, što znači da je kreiran na više kolona istovremeno. Ovaj indeks kombinuje GostID i StatusRezervacije, a namijenjen je upitima koji pretražuju rezervacije određenog gosta sa određenim statusom. Kompozitni indeksi su posebno efikasni kada se u WHERE dijelu upita koriste dve kolone.

```
CREATE INDEX IX_Rezervacije_GostStatus ON Rezervacije(GostID, StatusRezervacije);
```

Slika 20 - Postavljanje kompozitnog indeksa koji obuhvata više kolona istovremeno na kolonama "GostID" i "StatusRezervacije"

4.5. Kompozitni indeks na kolonama „TipSobe“ i „CijenaPoNoci“ u tabeli „Sobe“

Sedmi indeks je kompozitni indeks na kolonama TipSobe i CijenaPoNoci u tabeli Sobe. Ovaj indeks je važan za upite koji pretražuju sobe po tipu i cijeni, što je česta operacija prilikom online pretrage dostupnih soba.

```
CREATE INDEX IX_Sobe_TipCijena ON Sobe(TipSobe, CijenaPoNoci);
```

Slika 21 - Postavljanje kompozitnog indeksa koji obuhvata kolone TipSobe i CijenaPoNoci u tabeli "Sobe"

5. Unos podataka u bazu podataka

Nakon što je struktura baze podataka u potpunosti definisana, potrebno je unijeti test podatke koji će omogućiti demonstraciju funkcionalnosti sistema. Test podaci su odabrani tako da pokriju različite scenarije koji se mogu pojaviti u stvarnom radu sistema.

5.1. Unos podataka u tabelu „Gosti“

U tabelu Gosti unosi se pet gostiju sa različitim podacima. Marko Marković iz Beograda je gost koji je već pet puta boravio u hotelu, što će biti evidentirano kroz broj dolazaka. Ana Anić iz Zagreba ima dva dolaska. John Smith iz Londona dolazi prvi put. Milica Milić iz Banje Luke ima tri dolaska. Petar Petrović iz Novog Sada dolazi prvi put. Ovi podaci će omogućiti testiranje funkcije za izračunavanje popusta na osnovu broja dolazaka.

```
INSERT INTO Gosti (Ime, Prezime, Email, Telefon, Adresa, Grad, Drzava, BrojDolazaka) VALUES
('Marko', 'Marković', 'marko.markovic@gmail.com', '065 123 456', 'Kralja Petra 1', 'Beograd', 'Srbija', 5),
('Ana', 'Anić', 'ana.anic@yahoo.com', '066 789 012', 'Zagrebačka 15', 'Zagreb', 'Hrvatska', 2),
('John', 'Smith', 'john.smith@outlook.com', '+1 212 555 1234', 'Main Street 10', 'New York', 'USA', 1),
('Milica', 'Milić', 'milica.milic@gmail.com', '065 987 654', 'Vojvode Stepe 8', 'Banja Luka', 'BiH', 3),
('Petar', 'Petrović', 'petar.petrovic@hotmail.com', '066 234 567', 'Cara Dušana 22', 'Novi Sad', 'Srbija', 0),
('Hans', 'Müller', 'hans.mueller@gmail.com', '+49 151 12345678', 'Hauptstrasse 15', 'Berlin', 'Njemacka', 4),
('Emily', 'Johnson', 'emily.johnson@yahoo.com', '+1 310 555 6789', 'Sunset Blvd 45', 'Los Angeles', 'USA', 2),
('Nikola', 'Nikolić', 'nikolanikolic85@gmail.com', '065 111 223', 'Bulevar Kralja Aleksandra 7', 'Beograd', 'Srbija', 3),
('Lena', 'Schmidt', 'lena.schmidt@outlook.de', '+49 152 98765432', 'Bahnhofstrasse 8', 'Munchen', 'Njemacka', 1),
('Michael', 'Brown', 'michael.brown@hotmail.com', '+1 617 555 7890', 'Broadway 123', 'Boston', 'USA', 2),
('Jelena', 'Jovanović', 'jelenajovanovic@gmail.com', '066 333 444', 'Nemanjina 11', 'Niš', 'Srbija', 5),
('Thomas', 'Weber', 'thomas.weber@web.de', '+49 157 11223344', 'Ludwigstrasse 22', 'Hamburg', 'Njemacka', 0),
('Sarah', 'Davis', 'sarah.davis@protonmail.com', '+1 206 555 4321', 'Pike Street 78', 'Seattle', 'USA', 3),
('Ivan', 'Ivanović', 'ivan.ivanovic@yahoo.com', '065 555 666', 'Kralja Milana 4', 'Kragujevac', 'Srbija', 2),
('Laura', 'Fischer', 'laura.fischer@gmail.com', '+49 151 99887766', 'Goethestrasse 12', 'Frankfurt', 'Njemacka', 4),
('David', 'Wilson', 'dwilson83@outlook.com', '+1 415 555 1122', 'Market Street 50', 'San Francisco', 'USA', 1),
('Marija', 'Marić', 'marija.maric@hotmail.com', '066 777 888', 'Trg Republike 3', 'Podgorica', 'Crna Gora', 2),
('Andreas', 'Becker', 'a.becker@gmx.de', '+49 152 55443322', 'Schillerstrasse 9', 'Stuttgart', 'Njemacka', 0),
```

Slika 22 - Unos podataka u tabelu "Gosti"

5.2. Unos podataka u tabelu „Sobe“

U tabelu Sobe unosi se šest soba različitih tipova. Dvije single sobe (101 i 102) na prvom spratu, dvije double sobe (201 i 202) na drugom spratu, i dvije suite sobe (301 i 302) na trećem spratu. Statusi soba su različiti: neke su dostupne, jedna je zauzeta, jedna je rezervisana. Ovo omogućava testiranje upita koji prikazuju samo dostupne sobe.

```
INSERT INTO Sobe (BrojSobe, TipSobe, Kapacitet, CijenaPoNoci, Sprat, Pogled, StatusSobe) VALUES
('101', 'Single', 1, 55.00, 1, 'Gradski', 'Dostupna'),
('102', 'Single', 1, 55.00, 1, 'Dvorište', 'Dostupna'),
('103', 'Single', 1, 52.00, 1, 'Gradski', 'Zauzeta'),
('104', 'Single', 1, 52.00, 1, 'Dvorište', 'Dostupna'),
('105', 'Single', 1, 58.00, 1, 'Gradski', 'Dostupna'),
('106', 'Single', 1, 58.00, 1, 'Dvorište', 'Rezervisana'),
('107', 'Single', 1, 60.00, 1, 'Gradski', 'Dostupna'),
('108', 'Single', 1, 60.00, 1, 'Dvorište', 'Zauzeta'),
('109', 'Single', 1, 53.00, 1, 'Gradski', 'Dostupna'),
('110', 'Single', 1, 53.00, 1, 'Dvorište', 'Dostupna'),
('111', 'Double', 2, 85.00, 1, 'Gradski', 'Dostupna'),
('112', 'Double', 2, 85.00, 1, 'Dvorište', 'Dostupna'),
```

Slika 23 - Unos podataka u tabelu "Sobe"

5.3. Unos podataka u tabelu „Rezervacije“

U tabelu Rezervacije unosi se pet rezervacija koje pokrivaju različite periode i različite goste. Marko Marković ima dvije rezervacije, što će povećati njegov broj dolazaka na sedam prilikom kreiranja rezervacija kroz proceduru. Rezervacije su raspoređene u periodu od aprila do jula 2025. godine. Jedna rezervacija je označena kao "Na čekanju" kako bi se testiralo filtriranje po statusu.

```
INSERT INTO Rezervacije (GostID, SobaID, DatumPrijava, DatumOdjave, BrojOdraslih, BrojDjece, StatusRezervacije) VALUES
(1, 1, '2025-01-10', '2025-01-15', 1, 0, 'Potvrđena'),
(2, 2, '2025-01-12', '2025-01-18', 2, 0, 'Potvrđena'),
(3, 3, '2025-01-15', '2025-01-17', 1, 0, 'Otkazana'),
(4, 4, '2025-01-20', '2025-01-25', 2, 0, 'Potvrđena'),
(5, 5, '2025-01-22', '2025-01-28', 2, 1, 'Potvrđena'),
(6, 6, '2025-01-25', '2025-01-30', 2, 0, 'Potvrđena'),
(7, 7, '2025-01-28', '2025-02-02', 2, 0, 'Završena'),
(8, 8, '2025-02-01', '2025-02-05', 2, 1, 'Završena'),
(9, 9, '2025-02-03', '2025-02-08', 2, 0, 'Završena'),
(10, 10, '2025-02-05', '2025-02-10', 3, 0, 'Potvrđena'),
(11, 11, '2025-02-08', '2025-02-12', 2, 2, 'Završena'),
(12, 12, '2025-02-10', '2025-02-15', 2, 1, 'Završena'),
(13, 13, '2025-02-12', '2025-02-18', 2, 0, 'Potvrđena'),
(14, 14, '2025-02-15', '2025-02-20', 2, 0, 'Završena'),
```

Slika 24 - Unos podataka u tabelu "Rezervacije"

5.4. Unos podataka u tabelu „Usluge“

U tabelu Usluge unosi se pet različitih usluga koje hotel nudi. Doručak i večera spadaju u kategoriju hrane, wellness u kategoriju spa, parking u kategoriju parking, a room service u kategoriju usluga. Svaka usluga ima svoju cijenu, što će omogućiti testiranje izračunavanja ukupne cijene rezervacije sa uslugama.

```
INSERT INTO Usluge (NazivUsluge, Opis, Cijena, Kategorija) VALUES
('Doručak', 'Švedski sto sa domaćim specijalitetima', 15.00, 'Hrana'),
('Večera', 'Trodnevni meni po izboru', 25.00, 'Hrana'),
('Wellness', 'Korištenje spa centra 2 sata', 30.00, 'Spa'),
('Parking', 'Čuvani parking prostor', 10.00, 'Parking'),
('Room service', 'Usluga dostave u sobu', 5.00, 'Usluga'),
('Polupansion', 'Doručak i večera', 35.00, 'Hrana'),
('Puni pansion', 'Doručak, ručak i večera', 55.00, 'Hrana'),
('Ručak', 'Dnevni meni sa tri jela', 20.00, 'Hrana'),
('Domaća rakija', 'Čaša domaće šljivovice', 4.00, 'Piće'),
('Vino', 'Čaša domaćeg vina', 3.50, 'Piće'),
('Masaža leđa', 'Klasična masaža leđa 30 minuta', 35.00, 'Spa'),
('Masaža cijelog tijela', 'Relaks masaža 60 minuta', 60.00, 'Spa'),
('Sauna', 'Korištenje finske saune', 25.00, 'Spa'),
('Jacuzzi', 'Hidromasažna kada 45 minuta', 28.00, 'Spa'),
('Bazen', 'Korištenje bazena', 15.00, 'Spa'),
('Garaža', 'Podzemna garaža sa nadzorom', 15.00, 'Parking'),
('Aerodromski transfer', 'Prevoz od/do aerodroma', 40.00, 'Transport'),
('Dodatni krevet', 'Dodatni krevet u sobi', 25.00, 'Usluga'),
```

Slika 25 - Unos podataka u tabelu "Usluge"

5.5. Unos podataka u tabelu „RezervacijeUsluge“

U tabelu RezervacijeUsluge unose se veze između rezervacija i usluga. Prva rezervacija (Marko Marković, soba 101) uključuje doručak za pet dana i parking za pet dana. Druga rezervacija (Ana Anić, soba 201) uključuje doručak za pet dana i večeru za tri dana. Treća rezervacija (John Smith, soba 301) uključuje wellness dva puta. Ovi podaci omogućavaju testiranje izračunavanja ukupne cijene rezervacije.

```
INSERT INTO RezervacijeUsluge (RezervacijaID, UslugaID, Kolicina, CijenaPoJedinici) VALUES
(92, 1, 5, 15.00),
(92, 5, 5, 5.00),
(93, 1, 6, 15.00),
(93, 2, 4, 25.00),
(95, 1, 5, 15.00),
(111, 1, 4, 15.00),
(111, 5, 4, 5.00),
(112, 2, 5, 25.00),
(112, 16, 1, 25.00),
(113, 1, 5, 15.00),
(113, 17, 3, 12.00),
(116, 2, 5, 25.00),
(116, 20, 1, 35.00),
(117, 1, 4, 15.00),
(166, 3, 1, 30.00),
(166, 15, 2, 28.00),
(168, 17, 2, 12.00),
(169, 1, 5, 15.00),
(250, 16, 1, 25.00),
(231, 2, 5, 25.00),
(231, 25, 1, 45.00),
(259, 25, 1, 45.00),
(260, 3, 1, 30.00),
(260, 26, 1, 22.00),
(264, 30, 1, 25.00)
```

Slika 26 - Unos podataka u tabelu "RezervacijeUsluge"

5.6. Unos podataka u tabelu Placanja

U tabelu Placanja unose se evidentirane uplate za rezervacije. Prva rezervacija je plaćena karticom sa transakcijskim ID-em. Druga rezervacija je plaćena gotovinom, pa nema transakcijski ID. Treća rezervacija je plaćena PayPal-om. Četvrta rezervacija ima uplatu koja je još uvijek na čekanju.

```
INSERT INTO Placanja (RezervacijaID, Iznos, NacinPlacanja, StatusPlacanja, TransakcijaID) VALUES
(92, 325.00, 'Kartica', 'Potvrđeno', 'TXN00123456'),
(93, 470.00, 'Gotovina', 'Potvrđeno', NULL),
(94, 890.00, 'PayPal', 'Potvrđeno', 'PYPL987654'),
(95, 275.00, 'Kartica', 'Potvrđeno', 'TXN789012'),
(96, 520.00, 'Kartica', 'Potvrđeno', 'TXN11223344'),
(97, 180.00, 'Gotovina', 'Potvrđeno', NULL),
```

Slika 27 - Unos podataka u tabelu "Placanja"

6. Kreiranje pogleda (engl. Views) za jednostavniji pristup podacima

Pogledi su virtualne tabele koje ne čuvaju podatke fizički, već predstavljaju rezultat unaprijed definisanog upita. Oni pojednostavljuju pristup podacima tako što omogućavaju korisnicima da rade sa jednostavnom strukturom, skrivajući kompleksnost osnovnih tabela i relacija. Također, pogledi povećavaju sigurnost jer mogu prikazati samo određene kolone ili redove, skrivajući osjetljive podatke.

6.1. Pogled „vw_RezervacijeDetalji“

Prvi pogled, nazvan vw_RezervacijeDetalji, kreiran je da pruži sveobuhvatan pregled rezervacija sa svim relevantnim detaljima. Ovaj pogled spaja podatke iz tabela Rezervacije, Gosti i Sobe, prikazujući informacije koje su korisne za operativni rad recepcije i menadžmenta.

```
CREATE VIEW vw_RezervacijeDetalji AS
SELECT
    r.RezervacijaID,
    g.Ime + ' ' + g.Prezime AS ImeGosta,
    g.Email,
    g.Telefon,
    s.BrojSobe,
    s.TipSobe,
    s.CijenaPoNoci,
    r.DatumPrijava,
    r.DatumOdjave,
    DATEDIFF(DAY, r.DatumPrijava, r.DatumOdjave) AS BrojNoci,
    r.StatusRezervacije,
    (DATEDIFF(DAY, r.DatumPrijava, r.DatumOdjave) * s.CijenaPoNoci) AS OsnovnaCijena
FROM Rezervacije r
INNER JOIN Gosti g ON r.GostID = g.GostID
INNER JOIN Sobe s ON r.SobaID = s.SobaID;
GO

CREATE VIEW vw_PrihodiPoSobama AS
SELECT
    s.BrojSobe,
    s.TipSobe,
    COUNT(r.RezervacijaID) AS BrojRezervacija,
    ISNULL(SUM(p.Iznos), 0) AS UkupniPrihod,
    AVG(p.Iznos) AS ProsijecnaUplata
FROM Sobe s
LEFT JOIN Rezervacije r ON s.SobaID = r.SobaID
LEFT JOIN Placanja p ON r.RezervacijaID = p.RezervacijaID
GROUP BY s.BrojSobe, s.TipSobe;
```

Slika 28 - Query korišten za kreiranje pogleda "vw_RezervacijeDetalji"

Pogled prikazuje identifikator rezervacije, ime gosta (spojeno iz kolona Ime i Prezime), email i telefon gosta, broj sobe i tip sobe, cijenu po noći, datume prijave i odjave, broj noći (izračunat pomoću DATEDIFF funkcije), status rezervacije i osnovnu cijenu smještaja (broj noći pomnožen

sa cijenom po noći). Kreiranjem ovog pogleda, korisnici ne moraju svaki put pisati kompleksne JOIN upite da bi dobili potrebne informacije. Umjesto toga, mogu jednostavno izvršiti SELECT * FROM vw_RezervacijeDetalji i dobiti sve relevantne podatke.

6.2. Pogled “vw_PrihodiPoSobama”

Drugi pogled, nazvan vw_PrihodiPoSobama, kreiran je za potrebe finansijskog izvještavanja. Ovaj pogled prikazuje prihode ostvarene po svakoj sobi u hotelu. Spaja podatke iz tabela Sobe, Rezervacije i Placanja, grupirajući ih po sobama.

```
CREATE VIEW vw_PrihodiPoSobama AS
SELECT
    s.BrojSobe,
    s.TipSobe,
    COUNT(r.RezervacijaID) AS BrojRezervacija,
    ISNULL(SUM(p.Iznos), 0) AS UkupniPrihod,
    AVG(p.Iznos) AS ProsijecnaUplata
FROM Sobe s
LEFT JOIN Rezervacije r ON s.SobaID = r.SobaID
LEFT JOIN Placanja p ON r.RezervacijaID = p.RezervacijaID
GROUP BY s.BrojSobe, s.TipSobe;
```

Slika 29 - Query korišten za kreiranje pogleda "vw_PrihodiPoSobama"

Pogled prikazuje broj sobe, tip sobe, broj rezervacija koje su vezane za tu sobu, ukupni prihod (suma svih iznosa plaćanja) i prosječnu uplatu po rezervaciji. Korištenje funkcije ISNULL osigurava da sobe koje nikada nisu bile rezervisane budu prikazane sa nultim prihodom, a ne sa NULL vrijednošću. Ovaj pogled je posebno koristan za menadžment jer omogućava brz uvid u to koje sobe donose najviše prihoda, što može uticati na strategije određivanja cijena i promocije.

7. Kreiranje uskladištenih procedura za automatizaciju

Uskladištene procedure su prethodno kompajlirani SQL kod koji se izvršava na serveru. One omogućavaju ponovno korištenje koda, bolje performanse jer se SQL kod kompajlira jednom prilikom kreiranja procedure, a zatim se izvršava više puta bez ponovnog kompajliranja, te povećanu sigurnost jer se korisnicima može dati dozvola za izvršavanje procedure bez davanja direktnog pristupa tabelama.

7.1. Procedura “sp_KreirajRezervaciju”

Prva procedura, nazvana sp_KreirajRezervaciju, automatizuje proces kreiranja nove rezervacije. Ova procedura prihvata parametre koji definišu rezervaciju, provjerava ispravnost podataka, kreira rezervaciju, ažurira status sobe i ažurira broj dolazaka gosta. Cijela operacija je obavljena transakcijom, što znači da će se ili sve promjene primijeniti ili nijedna, u slučaju greške.

```
CREATE PROCEDURE sp_KreirajRezervaciju
    @GostID INT,
    @SobaID INT,
    @DatumPrijava DATE,
    @DatumOdjave DATE,
    @BrojOdraslih INT = 1,
    @BrojDjece INT = 0,
    @Napomena NVARCHAR(500) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        IF NOT EXISTS (SELECT 1 FROM Sobe WHERE SobaID = @SobaID AND StatusSobe = 'Dostupna')
        BEGIN
            RAISERROR('Soba nije dostupna za rezervaciju!', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        IF NOT EXISTS (SELECT 1 FROM Gosti WHERE GostID = @GostID)
        BEGIN
            RAISERROR('Gost ne postoji u bazi!', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        INSERT INTO Rezervacije (GostID, SobaID, DatumPrijava, DatumOdjave, BrojOdraslih, BrojDjece, Napomena)
        VALUES (@GostID, @SobaID, @DatumPrijava, @DatumOdjave, @BrojOdraslih, @BrojDjece, @Napomena);

        UPDATE Sobe SET StatusSobe = 'Rezervisana' WHERE SobaID = @SobaID;

        UPDATE Gosti SET BrojDolazaka = BrojDolazaka + 1 WHERE GostID = @GostID;

        COMMIT TRANSACTION;

        SELECT SCOPE_IDENTITY() AS RezervacijaID;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

Slika 30 - Procedura kreirana za automatizaciju kreiranja novih rezervacija

Procedura prihvata sljedeće parametre, GostID (identifikator gosta), SobaID (identifikator sobe), DatumPrijava, DatumOdjave, BrojOdraslih (podrazumijevano 1), BrojDece (podrazumijevano 0) i Napomena (opcionally).

Unutar procedure, prvo se provjerava da li je soba dostupna. Ako soba ne postoji ili nije u statusu “Dostupna”, procedura prekida izvršavanje sa odgovarajućom porukom o grešci. Zatim se provjerava da li gost postoji u bazi. Ako gost ne postoji, procedura također prekida izvršavanje. Nakon što su provjere prošle, procedura unosi novu rezervaciju u tabelu Rezervacije. Zatim ažurira status sobe u Rezervisana, kako bi se spriječilo duplo rezervisanje iste sobe. Na kraju, povećava broj dolazaka gosta za jedan, čime se prati lojalnost. Ako sve naredbe izvrše uspješno, transakcija se potvrđuje (COMMIT) i procedura vraća identifikator novokreirane rezervacije korištenjem funkcije SCOPE_IDENTITY(). U slučaju bilo kakve greške, transakcija se poništava (ROLLBACK) i greška se prosleđuje pozivaocu procedure.

7.2. Procedura “sp_IzracunajUkupnuCijenu”

Druga procedura, nazvana sp_IzracunajUkupnuCenu, izračunava ukupnu cijenu rezervacije uključujući smještaj i sve dodatne usluge. Ova procedura prihvata identifikator rezervacije kao parametar i vraća detaljan pregled cijene. Procedura prvo izračunava osnovnu cijenu smještaja. To čini tako što iz tabele Rezervacije dohvata datum prijave i datum odjave, a iz tabele Sobe dohvata cijenu po noći. Broj noći se izračunava pomoću DATEDIFF funkcije, a zatim se množi sa cijenom po noći. Zatim procedura izračunava ukupnu cijenu usluga. Ovo se radi tako što se iz tabele RezervacijeUsluge sumiraju proizvodi količine i cijene po jedinici za sve usluge vezane za datu rezervaciju. Funkcija ISNULL osigurava da će rezultat biti 0 ako nema dodatnih usluga. Na kraju, procedura sabira osnovnu cijenu smještaja i cijenu usluga, te vraća sve tri vrijednosti (osnovnu cijenu, cijenu usluga i ukupnu cijenu) kao rezultat.

```
CREATE PROCEDURE sp_IzracunajUkupnuCijenu
    @RezervacijaID INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @OsnovnaCijena DECIMAL(10,2);
    DECLARE @CijenaUsluga DECIMAL(10,2);
    DECLARE @UkupnaCijena DECIMAL(10,2);

    -- Izračunavanje osnovne cijene smještaja
    SELECT @OsnovnaCijena = DATEDIFF(DAY, r.DatumPrijava, r.DatumOdjave) * s.CijenaPoNoci
    FROM Rezervacije r
    INNER JOIN Sobe s ON r.SobaID = s.SobaID
    WHERE r.RezervacijaID = @RezervacijaID;

    -- Izračunavanje cijene usluga
    SELECT @CijenaUsluga = ISNULL(SUM(ru.Kolicina * ru.CijenaPoJedinici), 0)
    FROM RezervacijeUsluge ru
    WHERE ru.RezervacijaID = @RezervacijaID;

    SET @UkupnaCijena = @OsnovnaCijena + @CijenaUsluga;

    -- Prikaz rezultata
    SELECT
        @RezervacijaID AS RezervacijaID,
        @OsnovnaCijena AS OsnovnaCijenaSmjestaja,
        @CijenaUsluga AS CijenaUsluga,
        @UkupnaCijena AS UkupnaCijena;
END;
```

Slika 31 - Procedura za automatsko izračunavanje ukupne cijene rezervacije uzimajući u obzir smještaj i ostale usluge

7.3. Procedura “sp_IzvjestajPopunjenosti”

Treća procedura, nazvana sp_IzvjestajPopunjenosti, kreirana je za potrebe izvještavanja o popunjenosti hotela u određenom mjesecu i godini. Ova procedura je posebno korisna menadžmentu hotela jer omogućava uvid u to koliko su sobe bile zauzete u posmatranom periodu, što može poslužiti za planiranje kapaciteta, određivanje cijena i analizu sezonskih trendova.

```
CREATE PROCEDURE sp_IzvjestajPopunjenosti
    @Mjesec INT,
    @Godina INT
AS
BEGIN
    SELECT
        s.BrojSobe,
        s.TipSobe,
        COUNT(r.RezervacijaID) AS BrojRezervacija,
        SUM(DATEDIFF(DAY, r.DatumPrijava, r.DatumOdjave)) AS UkupnoNoci
    FROM Sobe s
    LEFT JOIN Rezervacije r ON s.SobaID = r.SobaID
        AND MONTH(r.DatumPrijava) = @Mjesec
        AND YEAR(r.DatumPrijava) = @Godina
    GROUP BY s.BrojSobe, s.TipSobe;
END;
```

Slika 32 - Procedura kreirana za potrebe izvještavanja o popunjenosti hotela u određenom mjesecu i godini

Procedura prihvata dva parametra, Mjesec (broj od 1 do 12) i Godina (četvorocifreni broj). Na osnovu ovih parametara, procedura generiše izvještaj za tačno određeni mjesec. Unutar procedure, vrši se spajanje tabele Sobe sa tabelom Rezervacije koristeći LEFT JOIN. LEFT JOIN je odabran namjerno jer želimo da prikazemo sve sobe, pa čak i one koje nisu imale nijednu rezervaciju u traženom periodu. Za sobe bez rezervacija, broj rezervacija i ukupan broj noći biće prikazani kao nula. Uslov spajanja nije samo na SobaID, već uključuje i filtriranje rezervacija po mjesecu i godini datuma prijave. Ovo je važno jer nas zanimaju samo rezervacije koje su započele u traženom periodu. AND MONTH(r.DatumPrijava) = @Mjesec AND YEAR(r.DatumPrijava) = @Godina osigurava da se u obzir uzmu samo rezervacije čiji datum prijave odgovara traženom mjesecu i godini. Procedura zatim grupira podatke po broju sobe i tipu sobe koristeći GROUP BY klauzulu. Za svaku sobu, COUNT(r.RezervacijaID) broji koliko je rezervacija bilo u tom mjesecu. Ukoliko soba nije imala rezervacija, COUNT funkcija će vratiti 0. SUM(DATEDIFF(DAY, r.DatumPrijava, r.DatumOdjave)) izračunava ukupan broj noći koje su sobe bile rezervisane. DATEDIFF funkcija računa razliku u danima između datuma prijave i datuma odjave, a zatim se te vrijednosti sumiraju za sve rezervacije te sobe u traženom mjesecu. Konačni rezultat procedure je tabela sa četiri kolone: BrojSobe (broj sobe), TipSobe (tip sobe), BrojRezervacija (broj rezervacija u traženom mjesecu) i UkupnoNoci (ukupan broj noći koliko je soba bila rezervisana). Ovaj izvještaj daje menadžmentu jasan uvid u popunjenost hotela i može poslužiti za dalje analize.

8. Kreiranje trigera za automatsko izvršavanje radnji

Trigeri su posebne vrste uskladištenih procedura koje se ne pozivaju eksplicitno, već se automatski izvršavaju kada se desi određeni događaj na tabeli, kao što su INSERT, UPDATE ili DELETE operacije. Trigeri se koriste za održavanje integriteta podataka, automatsko ažuriranje povezanih podataka, logiranje promjena i implementaciju složenih poslovnih pravila.

8.1. Tabela za praćenje promjena cijena soba

Prije kreiranja trigera, potrebno je kreirati tabelu koja će čuvati historiju promjena cijena soba. Ova tabela, nazvana SobeCijeneLog, sadrži identifikator log zapisa, identifikator sobe čija je cijena promijenjena, staru cijenu, novu cijenu, datum promjene i korisnika koji je izvršio promjenu. SUSER_NAME() funkcija automatski bilježi ime korisnika koji je izvršio promjenu.

```
CREATE TABLE SobeCijeneLog (  
    LogID INT IDENTITY(1,1) PRIMARY KEY,  
    SobaID INT,  
    StaraCijena DECIMAL(10,2),  
    NovaCijena DECIMAL(10,2),  
    DatumPromjene DATETIME DEFAULT GETDATE(),  
    Korisnik NVARCHAR(100) DEFAULT SUSER_NAME()  
);
```

Slika 33 - Tabela koja čuva historiju promjena cijena soba

8.2. Trigger „trg_OtkaziRezervaciju“

Prvi trigger, nazvan trg_OtkaziRezervaciju, aktivira se nakon što se izvrši UPDATE operacija nad tabelom Rezervacije. Njegova svrha je da automatski oslobodi sobu kada se rezervacija otkáže, čime se održava konzistentnost između statusa rezervacije i statusa sobe.

Trigger koristi specijalne tabele inserted i deleted koje SQL Server automatski kreira u kontekstu trigera. Tabela inserted sadrži nove vrijednosti nakon UPDATE operacije, dok tabela deleted sadrži stare vrijednosti prije UPDATE operacije.

Trigger prvo provjerava da li je UPDATE operacija uticala na kolonu StatusRezervacije, koristeći funkciju UPDATE(). Ako jeste, trigger spaja tabele Sobe, inserted i deleted kako bi pronašao rezervacije čiji je status promijenjen u Otkazana, a prethodni status nije bio Otkazana. Za svaku takvu rezervaciju, trigger ažurira status odgovarajuće sobe u Dostupna.

Ovaj trigger osigurava da se sobe koje su bile rezervisane, a čija je rezervacija naknadno otkazana, automatski vraćaju u stanje dostupnosti, bez potrebe za ručnom intervencijom.

```

CREATE TRIGGER trg_OtkaziRezervaciju
ON Rezervacije
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF UPDATE(StatusRezervacije)
    BEGIN
        UPDATE s
        SET s.StatusSobe = 'Dostupna'
        FROM Sobe s
        INNER JOIN inserted i ON s.SobaID = i.SobaID
        INNER JOIN deleted d ON i.RezervacijaID = d.RezervacijaID
        WHERE i.StatusRezervacije = 'Otkazana'
        AND d.StatusRezervacije != 'Otkazana';
    END
END;

```

Slika 34 - Query korišten za kreiranje trigeru "trg_OtkaziRezervaciju"

8.3. Triger „trg_LogCijeneSoba“

Drugi trigger, nazvan trg_LogCijeneSoba, aktivira se nakon UPDATE operacije nad tabelom Sobe. Njegova svrha je da evidentira svaku promjenu cijene sobe u log tabelu, omogućavajući kasniju analizu promjena cijena i reviziju. Trigger prvo provjerava da li je UPDATE operacija uticala na kolonu CenaPoNoci. Ako jeste, trigger spaja tabele inserted i deleted kako bi pronašao sobe čija je cijena promijenjena. Za svaku takvu sobu, trigger upisuje zapis u log tabelu sa starom cijenom (iz deleted tabele) i novom cijenom (iz inserted tabele). Ova funkcionalnost je važna iz više razloga. Prvo, omogućava praćenje istorije cijena, što može biti korisno za analizu poslovanja. Drugo, omogućava reviziju u slučaju da neko neovlašteno promijeni cijene.

```

CREATE TRIGGER trg_LogCijeneSoba
ON Sobe
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF UPDATE(CijenaPoNoci)
    BEGIN
        INSERT INTO SobeCijeneLog (SobaID, StaraCijena, NovaCijena)
        SELECT i.SobaID, d.CijenaPoNoci, i.CijenaPoNoci
        FROM inserted i
        INNER JOIN deleted d ON i.SobaID = d.SobaID
        WHERE i.CijenaPoNoci != d.CijenaPoNoci;
    END
END;

```

Slika 35 - Query korišten za kreiranje trigeru "trg_LogCijeneSoba"

8.4. Trigger „trg_ProvjeraDuplihRezervacija“

Trigger trg_ProvjeraDuplihRezervacija aktivira se umjesto INSERT operacije nad tabelom Rezervacije. Njegova svrha je da spriječi duple rezervacije iste sobe za preklapajući vremenski period.

Prvo, trigger provjerava da li u tabeli Rezervacije već postoji potvrđena rezervacija (StatusRezervacije = 'Potvrđena') za istu sobu (SobaID) koja se preklapa sa datumima rezervacije koja se unosi (DatumPrijava i DatumOdjava). Ako takva rezervacija postoji, trigger generiše grešku putem RAISERROR metode i zaustavlja unos.

```
CREATE TRIGGER trg_ProvjeraDuplihRezervacija
ON Rezervacije
INSTEAD OF INSERT
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM Rezervacije r
        INNER JOIN inserted i ON r.SobaID = i.SobaID
        WHERE r.DatumPrijava < i.DatumOdjava
        AND r.DatumOdjava > i.DatumPrijava
        AND r.StatusRezervacije = 'Potvrđena'
    )
    BEGIN
        RAISERROR('Soba je već rezervisana u tom periodu!', 16, 1);
        RETURN;
    END

    INSERT INTO Rezervacije (GostID, SobaID, DatumPrijava, DatumOdjava, BrojOdraslih, BrojDjece, Napomena)
    SELECT GostID, SobaID, DatumPrijava, DatumOdjava, BrojOdraslih, BrojDjece, Napomena
    FROM inserted;
END;
```

Slika 36 - Query korišten za kreiranje triggera "trg_ProvjeraDuplihRezervacija"

Ukoliko nema konflikata, trigger omogućava normalan unos nove rezervacije tako što preuzima podatke iz virtualne tabele inserted i upisuje ih u tabelu Rezervacije.

Ovaj trigger je ključan za očuvanje integriteta podataka, jer osigurava da se soba ne može rezervirati dva puta za isti vremenski period, čime se izbjegavaju problemi sa preklapanjem i olakšava upravljanje raspoloživošću soba.

9. Korištenje korisničkih funkcija za ponovno korištenje koda

Korisnički definisane funkcije (engl. User Defined Functions) su objekti koji primaju parametre, izvršavaju određenu logiku i vraćaju vrijednost. Za razliku od procedura, funkcije se mogu koristiti unutar SELECT upita, što ih čini vrlo fleksibilnim za ponovno korištenje koda.

9.1. Funkcija „fn_IzracunajPopust“

Prva funkcija, nazvana fn_IzracunajPopust, izračunava popust koji se odobrava gostu na osnovu broja njegovih prethodnih dolazaka. Ova funkcija prima broj dolazaka kao parametar, a vraća procenat popusta kao decimalni broj. Logika funkcije je jednostavna. Ako je gost dolazio deset ili više puta, odobrava mu se popust od 20 posto. Ako je dolazio pet ili više puta, popust je 10 posto. Ako je dolazio tri ili više puta, popust je 5 posto. U suprotnom, popust nije odobren (0 posto). Ova funkcija je implementirana kao skalarna funkcija, što znači da vraća jednu vrijednost. Može se koristiti unutar SELECT upita za izračunavanje popusta za svakog gosta pojedinačno.

```
CREATE FUNCTION fn_IzracunajPopust(@BrojDolazaka INT)
RETURNS DECIMAL(5,2)
AS
BEGIN
    DECLARE @Popust DECIMAL(5,2);

    IF @BrojDolazaka >= 10
        SET @Popust = 20.00;
    ELSE IF @BrojDolazaka >= 5
        SET @Popust = 10.00;
    ELSE IF @BrojDolazaka >= 3
        SET @Popust = 5.00;
    ELSE
        SET @Popust = 0.00;

    RETURN @Popust;
END;
```

Slika 37 - UDF za izračunavanje popusta

9.2. Funkcija „fn_RaspoloziveSobe“

Druga funkcija, nazvana fn_RaspoloziveSobe, prikazuje sve sobe koje su dostupne za rezervaciju u određenom vremenskom periodu. Ova funkcija prima datum početka i datum završetka kao

parametre, a vraća tabelu sa svim kolonama iz tabele Sobe koje zadovoljavaju uslove dostupnosti. Ova funkcija je implementirana kao tabelarna funkcija, što znači da vraća skup redova (tabelu). Implementacija koristi složenu logiku za određivanje dostupnosti sobe. Soba je dostupna ako je njen status Dostupna i ako ne postoji nijedna potvrđena rezervacija koja se preklapa sa traženim periodom. Uslov za preklapanje rezervacija je sljedeći, rezervacija se preklapa ako je njen datum prijave manji ili jednak traženom datumu odjave i njen datum odjave veći ili jednak traženom datumu prijave. Ovo pokriva sve moguće slučajeve preklapanja perioda. Ova funkcija je izuzetno korisna za online sistem rezervacija jer omogućava korisnicima da vide koje sobe su slobodne za željeni period.

```
CREATE FUNCTION fn_RaspoloziveSobe(@DatumOd DATE, @DatumDo DATE)
RETURNS TABLE
AS
RETURN
(
    SELECT s.*
    FROM Sobe s
    WHERE s.StatusSobe = 'Dostupna'
    AND NOT EXISTS (
        SELECT 1
        FROM Rezervacije r
        WHERE r.SobaID = s.SobaID
        AND r.StatusRezervacije = 'Potvrđena'
        AND ((r.DatumPrijave <= @DatumDo AND r.DatumOdjave >= @DatumOd))
    )
);
```

Slika 38 - UDF za prikaz dostupnih soba za rezervaciju

10. Primjeri DML naredbi za manipulaciju podacima

DML (engl. Data Manipulation Language) naredbe se koriste za manipulaciju podacima u bazi. To su osnovne operacije koje svaki korisnički interfejs izvršava nad bazom podataka. U ovom poglavlju prikazani su konkretni primjeri svih DML naredbi nad bazom HotelRezervacije.

10.1. SELECT naredba za dohvaćanje podataka

Primjer br.1 - Dohvaćanje svih gostiju sortiranih po prezimenu. Ovaj upit je koristan za pregled svih registrovanih gostiju.

1 `SELECT * FROM Gosti ORDER BY Prezime, Ime;`

146 % No issues found

	GostID	Ime	Prezime	Email	Telefon	Adresa	Grad	Drzava	DatumRegistracije	BrojDolazaka
1	62	Jason	Adams	jason.adams@yahoo.com	+1 614 555 2233	High Street 250	Columbus	USA	2026-03-30 18:23:36.040	2
2	123	Adriana	Adric	adriana.adric@protonmail.com	065 444 222	Obala Lazara Pecara 8	Bar	Crna Gora	2026-03-30 18:23:36.040	1
3	75	Aleksandra	Aleksandrovic	aleksandra.aleksandrovic@hotmail.com	066 444 333	Vojvode Mišica 23	Užice	Srbija	2026-03-30 18:23:36.040	4
4	32	Aleksandar	Aleksic	aleksandar.aleksic@yahoo.com	065 444 777	Stevana Nemanje 19	Leskovac	Srbija	2026-03-30 18:23:36.040	2
5	159	Nora	Andersen	nora.andersen@outlook.dk	+45 21234567	Strøget 15	Kopenhagen	Danska	2026-03-30 18:26:05.020	0
6	25	Ashley	Anderson	ashley.anderson@protonmail.com	+1 713 555 2233	Westheimer Rd 321	Houston	USA	2026-03-30 18:23:36.040	3
7	151	Ethan	Anderson	ethan.anderson@yahoo.com	+1 206 555 4321	Pike Place 85	Seattle	USA	2026-03-30 18:26:05.020	1
8	2	Ana	Anic	ana.anic@yahoo.com	066 789 012	Zagrebicka 15	Zagreb	Hrvatska	2026-03-30 18:23:36.040	2
9	220	Stefan	Arsenijevic	stefan.arsenijevic@gmail.com	066 111 555	Vojvode Mišica 41	Kragujevac	Srbija	2026-03-30 18:27:28.860	2
10	142	Luka	Babic	luka.babic@gmail.com	065 987 321	Fra Andrije Kacica 8	Split	Hrvatska	2026-03-30 18:26:05.020	2
11	210	Stjepan	Babic	stjepan.babic@yahoo.com	066 555 888	Ulica Franje Tudmana 25	Zadar	Hrvatska	2026-03-30 18:27:28.860	1
12	125	Megan	Bailey	megan.bailey@hotmail.com	+1 207 555 1122	Congress Street 150	Portland	USA	2026-03-30 18:23:36.040	2
13	80	Rachel	Baker	rachel.baker@outlook.com	+1 410 555 1122	Pratt Street 300	Baltimore	USA	2026-03-30 18:23:36.040	3
14	42	Katrin	Bauer	katrin.bauer@gmail.com	+49 151 11225533	Domstrasse 5	Bremen	Njemačka	2026-03-30 18:23:36.040	2
15	121	Moritz	Beck	moritz.beck@web.de	+49 152 66778822	Marktgasse 12	Regensburg	Njemačka	2026-03-30 18:23:36.040	2
16	18	Andreas	Becker	a.becker@gmx.de	+49 152 55443322	Schillerstrasse 9	Stuttgart	Njemačka	2026-03-30 18:23:36.040	0
17	189	Lena	Berg	lena.berg@telia.com	+46 702345678	Avenyn 35	Göteborg	Švedska	2026-03-30 18:26:05.020	4

Primjer br.2 - Dohvaćanje rezervacija sa ukupnom cijenom smještaja. Ovaj upit spaja podatke iz tri tabele i izračunava ukupnu cijenu na osnovu broja noći i cijene po noći.

1 `SELECT`
2 `r.RezervacijaID,`
3 `g.Ime + ' ' + g.Prezime AS Gost,`
4 `s.BrojSobe,`
5 `r.DatumPrijava,`
6 `r.DatumOdjave,`
7 `DATEDIFF(DAY, r.DatumPrijava, r.DatumOdjave) * s.CijenaPoNoci AS UkupnaCijenaSmjestaja`
8 `FROM Rezervacije r`
9 `INNER JOIN Gosti g ON r.GostID = g.GostID`
10 `INNER JOIN Sobe s ON r.SobaID = s.SobaID`
11 `WHERE r.StatusRezervacije = 'Potvrđena';`

146 % No issues found

	RezervacijaID	Gost	BrojSobe	DatumPrijava	DatumOdjave	UkupnaCijenaSmjestaja
1	92	Marko Markovic	101	2025-01-10	2025-01-15	275.00
2	265	Marko Markovic	104	2025-08-15	2025-08-20	260.00
3	266	Marko Markovic	204	2025-08-15	2025-08-20	440.00
4	93	Ana Anic	102	2025-01-12	2025-01-18	330.00
5	95	Milica Milic	104	2025-01-20	2025-01-25	260.00
6	267	Milica Milic	105	2025-07-01	2025-07-10	522.00
7	96	Petar Petrovic	105	2025-01-22	2025-01-28	348.00
8	97	Hans Müller	106	2025-01-25	2025-01-30	290.00
9	101	Michael Brown	110	2025-02-05	2025-02-10	265.00

Primjer br.3 - Prikaz prihoda po mjesecima. Ovaj upit grupiše uplate po godini i mjesecu i izračunava ukupan prihod za svaki period.

```
1  SELECT
2      YEAR(DatumPlacanja) AS Godina,
3      MONTH(DatumPlacanja) AS Mjesec,
4      SUM(Iznos) AS UkupniPrihod
5  FROM Placanja
6  WHERE StatusPlacanja = 'Potvrđeno'
7  GROUP BY YEAR(DatumPlacanja), MONTH(DatumPlacanja)
8  ORDER BY Godina, Mjesec;
```

146 % No issues found

Results Messages

	Godina	Mjesec	UkupniPrihod
1	2026	3	75425.00

10.2. INSERT naredba za unos novih podataka

```
INSERT INTO Placanja (RezervacijaID, Iznos, NacinPlacanja, StatusPlacanja, TransakcijaID) VALUES
(92, 325.00, 'Kartica', 'Potvrđeno', 'TXN00123456'),
(93, 470.00, 'Gotovina', 'Potvrđeno', NULL),
(94, 890.00, 'PayPal', 'Potvrđeno', 'PYPL987654');
```

10.3. UPDATE naredba za izmjena postojećih podataka

UPDATE naredba se koristi za izmjenu postojećih podataka u tabelama. U nastavku je prikazan primjer otkazivanja rezervacije.

```
1  UPDATE Rezervacije
2  SET StatusRezervacije = 'Otkazana'
3  WHERE RezervacijaID = 92;
```

5 % No issues found

Messages

(1 row affected)

Completion time: 2026-04-02T20:57:34.6773330+02:00

	RezervacijaID	GostID	SobaID	DatumPrijava	DatumOdjave	DatumRezervacije	BrojOdraslih	BrojDjece	StatusRezervacije	Napomena
1	92	1	1	2025-01-10	2025-01-15	2026-03-30 19:14:33.783	1	0	Potvrdena	NULL
2	93	2	2	2025-01-12	2025-01-18	2026-03-30 19:14:33.783	2	0	Potvrdena	NULL
3	94	3	3	2025-01-15	2025-01-17	2026-03-30 19:14:33.783	1	0	Otkazana	NULL
4	95	4	4	2025-01-20	2025-01-25	2026-03-30 19:14:33.783	2	0	Potvrdena	NULL
5	96	5	5	2025-01-22	2025-01-28	2026-03-30 19:14:33.783	2	1	Potvrdena	NULL

Slika 39 - Tabela Rezervacije prije korištenja UPDATE naredbe

	RezervacijaID	GostID	SobaID	DatumPrijava	DatumOdjave	DatumRezervacije	BrojOdraslih	BrojDjece	StatusRezervacije	Napomena
1	92	1	1	2025-01-10	2025-01-15	2026-03-30 19:14:33.783	1	0	Otkazana	NULL
2	93	2	2	2025-01-12	2025-01-18	2026-03-30 19:14:33.783	2	0	Potvrdena	NULL
3	94	3	3	2025-01-15	2025-01-17	2026-03-30 19:14:33.783	1	0	Otkazana	NULL
4	95	4	4	2025-01-20	2025-01-25	2026-03-30 19:14:33.783	2	0	Potvrdena	NULL
5	96	5	5	2025-01-22	2025-01-28	2026-03-30 19:14:33.783	2	1	Potvrdena	NULL
6	97	6	6	2025-01-25	2025-01-30	2026-03-30 19:14:33.783	2	0	Potvrdena	NULL

Slika 40 - Tabela Rezervacije nakon korištenja UPDATE naredbe na ID 92

10.4. DELETE naredba za brisanje podataka

DELETE naredba se koristi za brisanje redova iz tabela. U nastavku je prikazan primjer brisanja usluge koja više nije u ponudi.

-- Brisanje usluge koja više nije u ponudi

DELETE FROM Usluge WHERE UslugaID = 5;

11. Primjer DDL naredbi za definisanje strukture

DDL (engl. Data Definition Language) naredbe se koriste za definisanje i izmjenu strukture baze podataka. Za razliku od DML naredbi koje manipulišu podacima, DDL naredbe definišu objekte u bazi.

11.1. ALTER naredba za izmjenu strukture

ALTER naredba se koristi za izmjenu postojećih objekata u bazi podataka. U nastavku su prikazani primjeri dodavanja novih kolona, mijenjanja tipova podataka i dodavanja ograničenja.

Primjer br.1 - Dodavanje nove kolone u tabelu Gosti. Ova kolona će čuvati dodatne napomene o gostu.

```
ALTER TABLE Gosti  
ADD Napomena NVARCHAR(500);
```

Primjer br.2 - Dodavanje UNIQUE ograničenja na postojeću kolonu. Ovo osigurava da broj telefona bude jedinstven za svakog gosta.

```
ALTER TABLE Gosti  
ADD CONSTRAINT UQ_Gosti_Telefon UNIQUE (Telefon);
```

11.2. DROP naredba za brisanje objekata

DROP naredba se koristi za brisanje objekata iz baze podataka. Ovo treba raditi oprezno jer se podaci trajno gube.

Primjer br.1 - Brisanje indeksa koji više nije potreban.

```
DROP INDEX IX_Sobe_TipCijena ON Sobe;
```

Primjer br.2 - Brisanje ograničenja koji više nije potreban.

```
ALTER TABLE Placanja  
DROP CONSTRAINT CHK_NacinPlacanja;
```

11.3. CREATE naredba za kreiranje objekata

CREATE naredba se koristi za kreiranje novih objekata u bazi podataka.

Primjer br.1 - Kreiranje indeksa za optimizaciju upita.

```
CREATE INDEX IX_Placanja_Datum ON Placanja(DatumPlacanja);
```

Primjer br.2 - Kreiranje nove tabele za čuvanje istorije cijena.

```
CREATE TABLE IstorijaCijena (  
    IstorijaID INT IDENTITY(1,1) PRIMARY KEY,  
    SobaID INT NOT NULL,  
    StaraCijena DECIMAL(10,2),  
    NovaCijena DECIMAL(10,2),  
    DatumPromjene DATETIME DEFAULT GETDATE(),  
    CONSTRAINT FK_Istorija_Sobe FOREIGN KEY (SobaID) REFERENCES Sobe(SobaID)  
);
```

Slika 41 - Query korišten za kreiranje nove tabele koja čuva istoriju cijena

11.4. TRUNCATE naredba za brzo brisanje svih podataka

TRUNCATE je DDL naredba koja briše sve redove iz tabele, ali zadržava strukturu tabele. Brža je od DELETE jer ne zapisuje svaki red pojedinačno.

Primjer br.1 - Brisanje svih zapisa iz tabele Sobe

TRUNCATE TABLE Sobe

Razlika između DELETE i TRUNCATE:

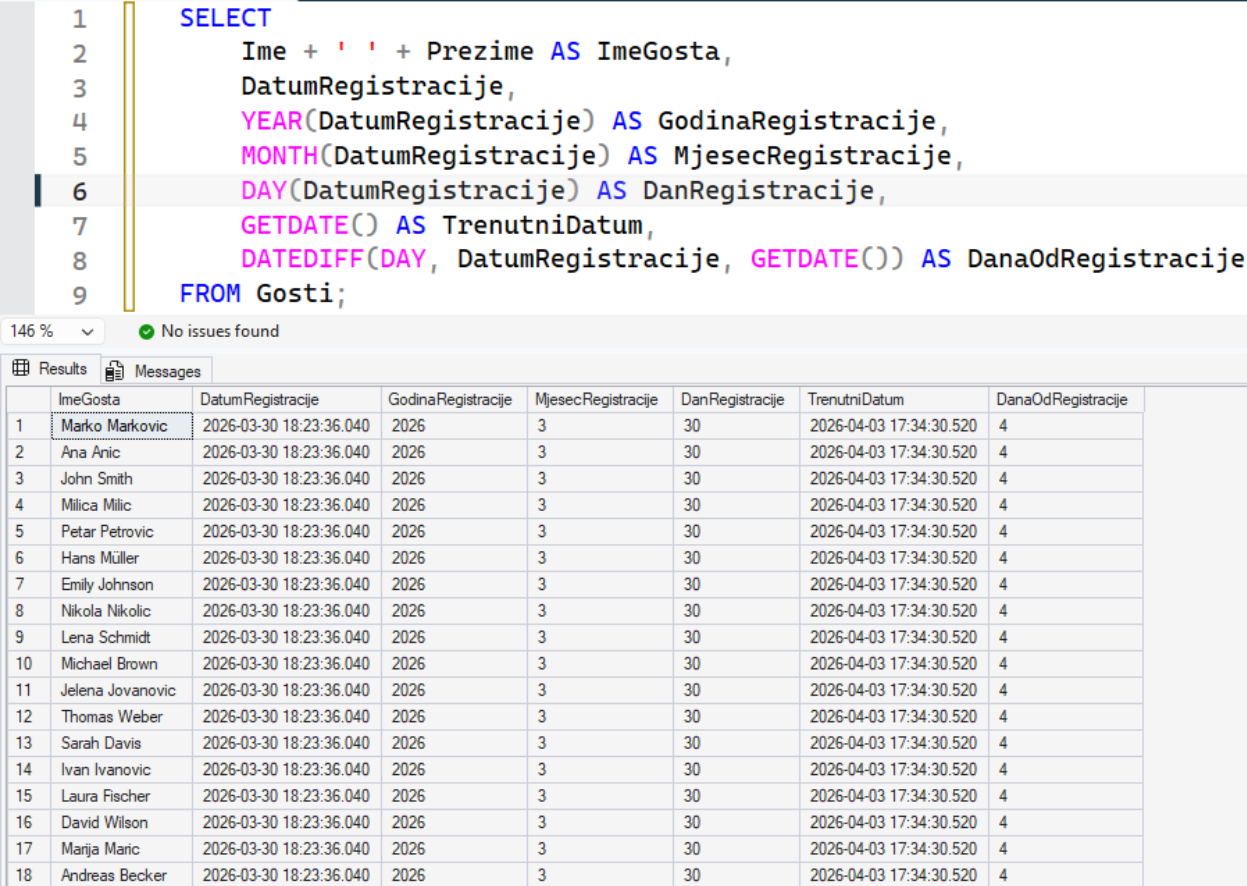
Karakteristika	DELETE	TRUNCATE
Brzina	Sporija	Veoma brza
Zapisivanje	Zapisuje svaki red	Zapisuje samo oslobađanje stranica
WHERE upit	Može se koristiti	Ne može se koristiti
Okidanje trigera	Aktivira trigere	Ne aktivira trigere
Resetovanje IDENTITY	Ne resetuje	Resetuje na početnu vrijednost

12. Primjeri korištenja različitih tipova funkcija

SQL Server nudi bogat skup ugrađenih funkcija koje se mogu koristiti u upitima. U ovom poglavlju prikazani su primjeri korištenja datumskih, string, agregatnih, numeričkih i funkcija opšte namjene.

12.1. Datumske funkcije

Datumske funkcije se koriste za rad sa datumima i vremenom. U nastavku je prikazan primjer koji za svakog gosta prikazuje godinu, mjesec i dan registracije, kao i broj dana od registracije do danas.



```
1 SELECT
2     Ime + ' ' + Prezime AS ImeGosta,
3     DatumRegistracije,
4     YEAR(DatumRegistracije) AS GodinaRegistracije,
5     MONTH(DatumRegistracije) AS MjesecRegistracije,
6     DAY(DatumRegistracije) AS DanRegistracije,
7     GETDATE() AS TrenutniDatum,
8     DATEDIFF(DAY, DatumRegistracije, GETDATE()) AS DanaOdRegistracije
9 FROM Gosti;
```

	ImeGosta	DatumRegistracije	GodinaRegistracije	MjesecRegistracije	DanRegistracije	TrenutniDatum	DanaOdRegistracije
1	Marko Markovic	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
2	Ana Anic	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
3	John Smith	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
4	Milica Milic	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
5	Petar Petrovic	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
6	Hans Müller	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
7	Emily Johnson	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
8	Nikola Nikolic	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
9	Lena Schmidt	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
10	Michael Brown	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
11	Jelena Jovanovic	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
12	Thomas Weber	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
13	Sarah Davis	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
14	Ivan Ivanovic	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
15	Laura Fischer	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
16	David Wilson	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
17	Marja Maric	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4
18	Andreas Becker	2026-03-30 18:23:36.040	2026	3	30	2026-04-03 17:34:30.520	4

Slika 42 - Primjer korištenja datumskih funkcija

12.2. String funkcije

String funkcije se koriste za manipulaciju tekstualnim podacima. U nastavku je prikazan primjer koji demonstrira konverziju velikih i malih slova, izračunavanje dužine stringa, te izdvajanje korisničkog imena i domena iz email adrese.

```

1  SELECT
2      Ime,
3      Prezime,
4      UPPER(Ime) AS ImeVelikaSlova,
5      LOWER(Prezime) AS PrezimeMalaSlova,
6      LEN(Ime + ' ' + Prezime) AS DužinaImena,
7      LEFT(Email, CHARINDEX('@', Email)-1) AS KorisnickoIme,
8      RIGHT(Email, LEN(Email) - CHARINDEX('@', Email)) AS Domen
9  FROM Gosti;

```

146 % No issues found

	Ime	Prezime	ImeVelikaSlova	PrezimeMalaSlova	DužinaImena	KorisnickoIme	Domen
1	Marko	Markovic	MARKO	markovic	14	marko.markovic	gmail.com
2	Ana	Anic	ANA	anic	8	ana.anic	yahoo.com
3	John	Smith	JOHN	smith	10	john.smith	outlook.com
4	Milica	Milic	MILICA	milic	12	milica.milic	gmail.com
5	Petar	Petrovic	PETAR	petrovic	14	petar.petrovic	hotmail.com
6	Hans	Müller	HANS	müller	11	hans.mueller	gmail.com

Slika 43 - Korištenje String funkcija za manipulaciju teksta

12.3. Agregatne funkcije

Agregatne funkcije se koriste za izračunavanje statističkih vrijednosti nad skupovima podataka. U nastavku je prikazan primjer koji izračunava ukupan broj gostiju, prosječan broj dolazaka, maksimalan i minimalan broj dolazaka, te ukupan broj dolazaka svih gostiju.

```

1  SELECT
2      COUNT(*) AS UkupnoGostiju,
3      AVG(BrojDolazaka) AS ProsjecanBrojDolazaka,
4      MAX(BrojDolazaka) AS MaxDolazaka,
5      MIN(BrojDolazaka) AS MinDolazaka,
6      SUM(BrojDolazaka) AS UkupnoDolazaka
7  FROM Gosti;

```

146 % No issues found

	UkupnoGostiju	ProsjecanBrojDolazaka	MaxDolazaka	MinDolazaka	UkupnoDolazaka
1	227	2	7	0	570

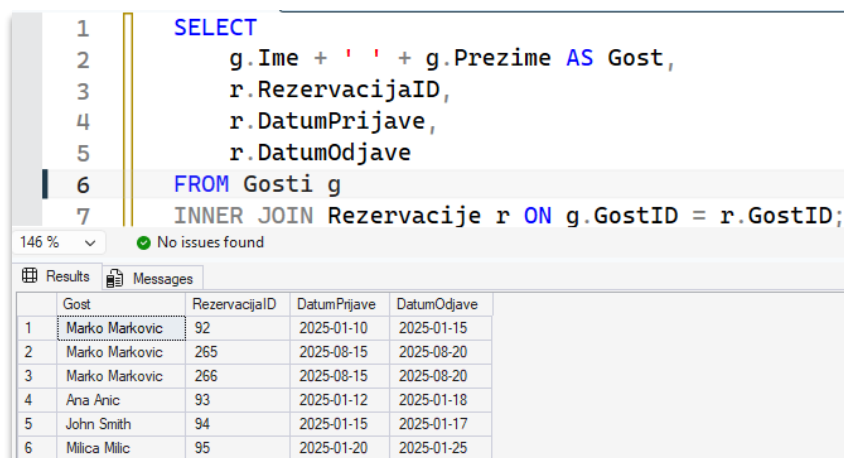
Slika 44 - Primjer korištenja više agregatnih funkcija

13. Primjeri JOIN upita za kombinovanje podataka iz više tabela

JOIN operacije se koriste za kombinovanje podataka iz dvije ili više tabela na osnovu veze između njih. U relacionim bazama podataka, podaci su često raspoređeni po različitim tabelama, a JOIN operacije omogućavaju njihovo povezivanje u smislene cijeline.

13.1. INNER JOIN

INNER JOIN vraća samo one redove koji imaju podudaranje u obe tabele koje se spajaju. Ovo je najčešće korištena vrsta JOIN-a. U primjeru ispod, prikazani su samo gosti koji imaju barem jednu rezervaciju.



```
1 SELECT
2     g.Ime + ' ' + g.Prezime AS Gost,
3     r.RezervacijaID,
4     r.DatumPrijave,
5     r.DatumOdjave
6 FROM Gosti g
7 INNER JOIN Rezervacije r ON g.GostID = r.GostID;
```

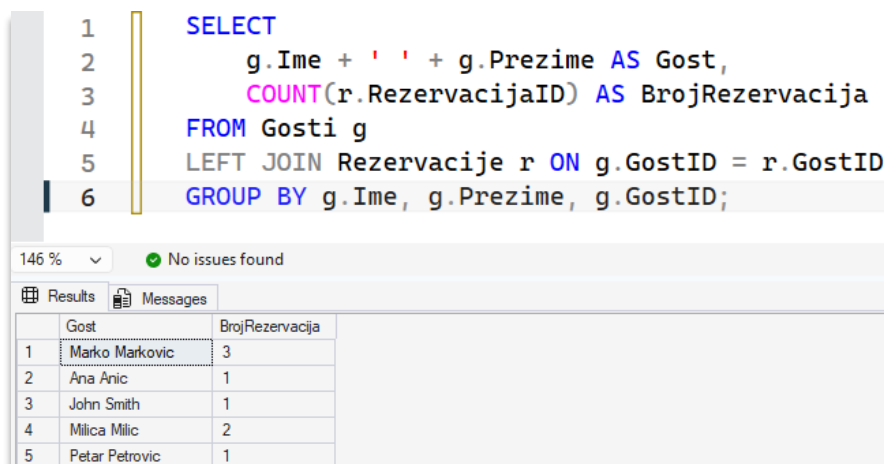
146 % No issues found

	Gost	RezervacijaID	DatumPrijave	DatumOdjave
1	Marko Markovic	92	2025-01-10	2025-01-15
2	Marko Markovic	265	2025-08-15	2025-08-20
3	Marko Markovic	266	2025-08-15	2025-08-20
4	Ana Anic	93	2025-01-12	2025-01-18
5	John Smith	94	2025-01-15	2025-01-17
6	Milica Milic	95	2025-01-20	2025-01-25

Slika 45 - Primjer korištenja INNER JOIN-a

13.2. LEFT JOIN (LEFT OUTER JOIN)

LEFT JOIN vraća sve redove iz lijeve tabele, a samo podudarajuće iz desne tabele. Ako nema podudaranja, kolone iz desne tabele će biti NULL. U primjeru ispod, prikazani su svi gosti, bez obzira da li imaju rezervacije ili ne.



```
1 SELECT
2     g.Ime + ' ' + g.Prezime AS Gost,
3     COUNT(r.RezervacijaID) AS BrojRezervacija
4 FROM Gosti g
5 LEFT JOIN Rezervacije r ON g.GostID = r.GostID
6 GROUP BY g.Ime, g.Prezime, g.GostID;
```

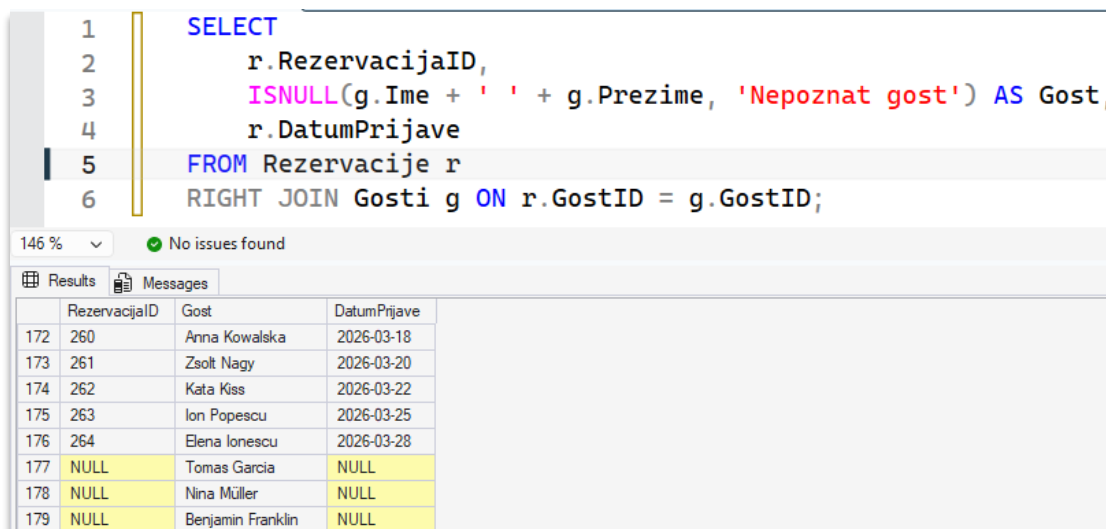
146 % No issues found

	Gost	BrojRezervacija
1	Marko Markovic	3
2	Ana Anic	1
3	John Smith	1
4	Milica Milic	2
5	Petar Petrovic	1

Slika 46 - Primjer korištenja LEFT JOIN-a

13.3. RIGHT JOIN (RIGHT OUTER JOIN)

RIGHT JOIN vraća sve redove iz desne tabele, a samo podudarajuće iz lijeve tabele. Ovo je simetrično LEFT JOIN-u. U primjeru ispod, prikazane su sve rezervacije, sa podacima o gostima ako postoje.



```
1 SELECT
2     r.RezervacijaID,
3     ISNULL(g.Ime + ' ' + g.Prezime, 'Nepoznat gost') AS Gost,
4     r.DatumPrijava
5 FROM Rezervacije r
6 RIGHT JOIN Gosti g ON r.GostID = g.GostID;
```

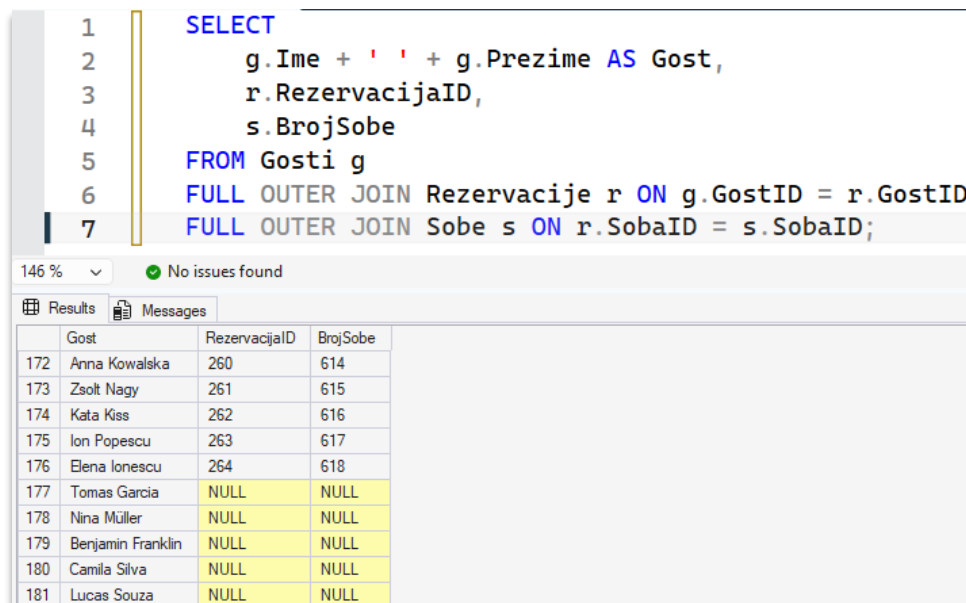
146 % No issues found

	RezervacijaID	Gost	DatumPrijava
172	260	Anna Kowalska	2026-03-18
173	261	Zsolt Nagy	2026-03-20
174	262	Kata Kiss	2026-03-22
175	263	Ion Popescu	2026-03-25
176	264	Elena Ionescu	2026-03-28
177	NULL	Tomas Garcia	NULL
178	NULL	Nina Müller	NULL
179	NULL	Benjamin Franklin	NULL

Slika 47 - Primjer korištenja RIGHT JOIN-a

13.4. FULL OUTER JOIN

FULL OUTER JOIN vraća sve redove iz obje tabele, spajajući gdje postoji podudaranje. Tamo gdje nema podudaranja, kolone iz suprotne tabele će biti NULL. U primjeru ispod, prikazane su sve kombinacije gostiju, rezervacija i soba.



```
1 SELECT
2     g.Ime + ' ' + g.Prezime AS Gost,
3     r.RezervacijaID,
4     s.BrojSobe
5 FROM Gosti g
6 FULL OUTER JOIN Rezervacije r ON g.GostID = r.GostID
7 FULL OUTER JOIN Sobe s ON r.SobaID = s.SobaID;
```

146 % No issues found

	Gost	RezervacijaID	BrojSobe
172	Anna Kowalska	260	614
173	Zsolt Nagy	261	615
174	Kata Kiss	262	616
175	Ion Popescu	263	617
176	Elena Ionescu	264	618
177	Tomas Garcia	NULL	NULL
178	Nina Müller	NULL	NULL
179	Benjamin Franklin	NULL	NULL
180	Camila Silva	NULL	NULL
181	Lucas Souza	NULL	NULL

Slika 48 - Primjer korištenja FULL OUTER JOIN-a

13.5. CROSS JOIN

CROSS JOIN vraća kartezijev proizvod dvije tabele, što znači da se svaki red iz prve tabele spaja sa svakim redom iz druge tabele. Ovo se rijetko koristi u praksi jer može proizvesti ogroman broj redova. U primjeru ispod, prikazane su sve moguće kombinacije gostiju i soba.

```
1 SELECT
2   g.Ime + ' ' + g.Prezime AS Gost,
3   s.BrojSobe
4 FROM Gosti g
5 CROSS JOIN Sobe s;
```

	Gost	BrojSobe
1	Marko Markovic	101
2	Ana Anic	101
3	John Smith	101
4	Milica Milic	101
5	Petar Petrovic	101
6	Hans Müller	101
7	Emily Johnson	101
8	Nikola Nikolic	101
9	Lena Schmidt	101
10	Michael Brown	101
11	Jelena Jovanovic	101
12	Thomas Weber	101
13	Sarah Davis	101

Query executed successfully. localhost\SQLEXPRESS (16.0 ... IVAN-PC\Admin (72) HotelRezervacije 00:00:00 Row: 1, Col: 1 39,271 rows

Slika 49 - Primjer korištenja CROSS JOIN-a

13.6. SELF JOIN

SELF JOIN je spajanje tabele same sa sobom. Koristi se kada treba uporediti redove unutar iste tabele. U primjeru ispod, prikazani su parovi gostiju koji žive u istom gradu.

```
1 SELECT
2   a.Ime + ' ' + a.Prezime AS Gost1,
3   b.Ime + ' ' + b.Prezime AS Gost2,
4   a.Grad
5 FROM Gosti a
6 INNER JOIN Gosti b ON a.Grad = b.Grad AND a.GostID < b.GostID;
```

	Gost1	Gost2	Grad
1	Marko Markovic	Nikola Nikolic	Beograd
2	Petar Petrovic	Vladimir Vladimirovic	Novi Sad
3	Marko Markovic	Sandra Savic	Beograd
4	Nikola Nikolic	Sandra Savic	Beograd
5	Christian Schulz	Lukas Hoffmann	Koln
6	Petar Petrovic	Ivana Ilic	Novi Sad
7	Vladimir Vladimirovic	Ivana Ilic	Novi Sad

Slika 50 - Primjer korištenja SELF JOIN-a

14. Kreiranje rezervne kopije baze podataka

Redovno kreiranje rezervnih kopija (engl. backup) predstavlja jednu od najvažnijih aktivnosti u administraciji baza podataka. Backup omogućava oporavak sistema u slučaju gubitka podataka usljed greške korisnika, kvara hardvera, softverskih problema ili sigurnosnih incidenata.

Sistem za upravljanje bazama podataka Microsoft SQL Server omogućava više vrsta backup-a koje se koriste u zavisnosti od potreba sistema i učestalosti promjena podataka.

14.1. Kreiranje potpunog backup-a (Full Backup)

Potpuni backup kreira kompletnu kopiju cijele baze podataka, uključujući sve tabele, indekse, procedure i podatke. Ovo je osnovni tip backup-a i predstavlja polaznu tačku za sve ostale vrste backup-a.

U naredbi BACKUP DATABASE navodi se ime baze koju želimo da backup-ujemo, dok se pomoću opcije TO DISK definiše lokacija na kojoj će backup fajl biti sačuvan.

Opcije koje se koriste:

- FORMAT – briše postojeći sadržaj i kreira novi backup set
- MEDIANAME – naziv medija
- NAME – naziv backup seta
- DESCRIPTION – opis backup-a
- STATS – prikaz napretka u procentima

```
1  BACKUP DATABASE HotelRezervacije
2  TO DISK = 'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\HotelRezervacije.bak'
3  WITH FORMAT,
4      MEDIANAME = 'HotelRezervacije_Backup',
5      NAME = 'Full Backup of HotelRezervacije',
6      DESCRIPTION = 'Potpuna rezervna kopija baze HotelRezervacije',
7      STATS = 10;
8  GO
```

% No issues found

Messages

```
10 percent processed.
20 percent processed.
30 percent processed.
40 percent processed.
50 percent processed.
60 percent processed.
70 percent processed.
80 percent processed.
90 percent processed.
100 percent processed.
Processed 760 pages for database 'HotelRezervacije', file 'HotelRezervacije' on file 1.
Processed 1 pages for database 'HotelRezervacije', file 'HotelRezervacije_log' on file 1.
BACKUP DATABASE successfully processed 761 pages in 0.058 seconds (102.395 MB/sec).
```

Slika 51 - Primjer uspješnog backup-a cijelokupne baze podataka

14.2. Druge vrste backup-a

Pored potpunog backup-a, SQL Server omogućava i dodatne vrste rezervnih kopija koje optimizuju prostor i vrijeme backup-a.

14.2.1. Diferencijalni backup (Differential Backup)

Diferencijalni backup čuva samo promjene koje su nastale nakon posljednjeg potpunog backup-a.

```
BACKUP DATABASE HotelRezervacije  
TO DISK = 'D:\SQLBackups\HotelRezervacije_Diff.bak'  
WITH DIFFERENTIAL,  
    STATS = 10;  
GO
```

Slika 52 - Primjer korištenja diferencijalnog backup-a

14.2.2. Backup loga transakcija (Transaction Log Backup)

Ova vrsta backup-a čuva sve transakcije koje su izvršene nad bazom. Omogućava vraćanje baze na tačno određeni trenutak (point-in-time recovery).

```
BACKUP LOG HotelRezervacije  
TO DISK = 'D:\SQLBackups\HotelRezervacije_Log.trn'  
WITH STATS = 10;  
GO
```

Slika 53 - Primjer korištenja Transaction Log Backup-a

14.2.3. Copy-only backup

Koristi se kada želimo napraviti backup bez uticaja na postojeći backup lanac.

```
BACKUP DATABASE HotelRezervacije  
TO DISK = 'D:\SQLBackups\HotelRezervacije_Copy.bak'  
WITH COPY_ONLY,  
    STATS = 10;  
GO
```

Slika 54 - Primjer korištenja copy-only backup-a

14.3. Restauracija baze iz backup-a

U slučaju gubitka podataka ili potrebe za vraćanjem prethodnog stanja, koristi se naredba RESTORE DATABASE.

Opcija REPLACE omogućava prepisivanje postojeće baze, dok STATS prikazuje napredak procesa.

```
RESTORE DATABASE HotelRezervacije
FROM DISK = 'D:\SQLBackups\HotelRezervacije_Full.bak'
WITH REPLACE,
     STATS = 10;
GO
```

Slika 55 - Primjer restauracije baze iz backup-a

15. ER dijagram i opis relacija

ER (Entity-Relationship) dijagram predstavlja grafički prikaz strukture baze podataka, odnosno način na koji su podaci organizovani i međusobno povezani. Pomoću ovog dijagrama moguće je lakše razumjeti kako sistem funkcioniše i kako se podaci kreću kroz aplikaciju.

U okviru baze podataka HotelRezervacije definisano je više entiteta koji zajedno čine cjelinu sistema za upravljanje hotelskim poslovanjem.

15.1. Entiteti i njihovi atributi

Entitet GOSTI koristi se za čuvanje podataka o korisnicima hotela. Svaki gost ima svoj jedinstveni identifikator (GostID), a uz to se čuvaju osnovni lični i kontakt podaci, kao i informacije poput datuma registracije i broja dolazaka. Ovaj entitet omogućava praćenje istorije gostiju i njihovih aktivnosti.

Entitet SOBE predstavlja sve smještajne jedinice u hotelu. Pored osnovnih informacija kao što su broj i tip sobe, evidentiraju se i kapacitet, cijena po noći, sprat i pogled. Takođe, atribut statusa sobe omogućava uvid u njenu trenutnu dostupnost.

Entitet REZERVACIJE ima centralnu ulogu u sistemu jer povezuje goste i sobe. Svaka rezervacija sadrži podatke o periodu boravka, datumu kreiranja rezervacije, broju osoba, kao i statusu same rezervacije. Na ovaj način moguće je precizno pratiti sve rezervacije u sistemu.

Entitet USLUGE obuhvata dodatne usluge koje hotel nudi, kao što su doručak, spa ili parking. Za svaku uslugu evidentira se naziv, opis, cijena i kategorija, što omogućava lakšu organizaciju i pregled ponude.

Veza između rezervacija i usluga realizovana je preko posebnog entiteta REZERVACIJE_USLUGE. Ova tabela omogućava da se jednoj rezervaciji dodijeli više usluga, uz evidentiranje količine i cijene po jedinici, čime se dobija fleksibilan model za obračun dodatnih troškova.

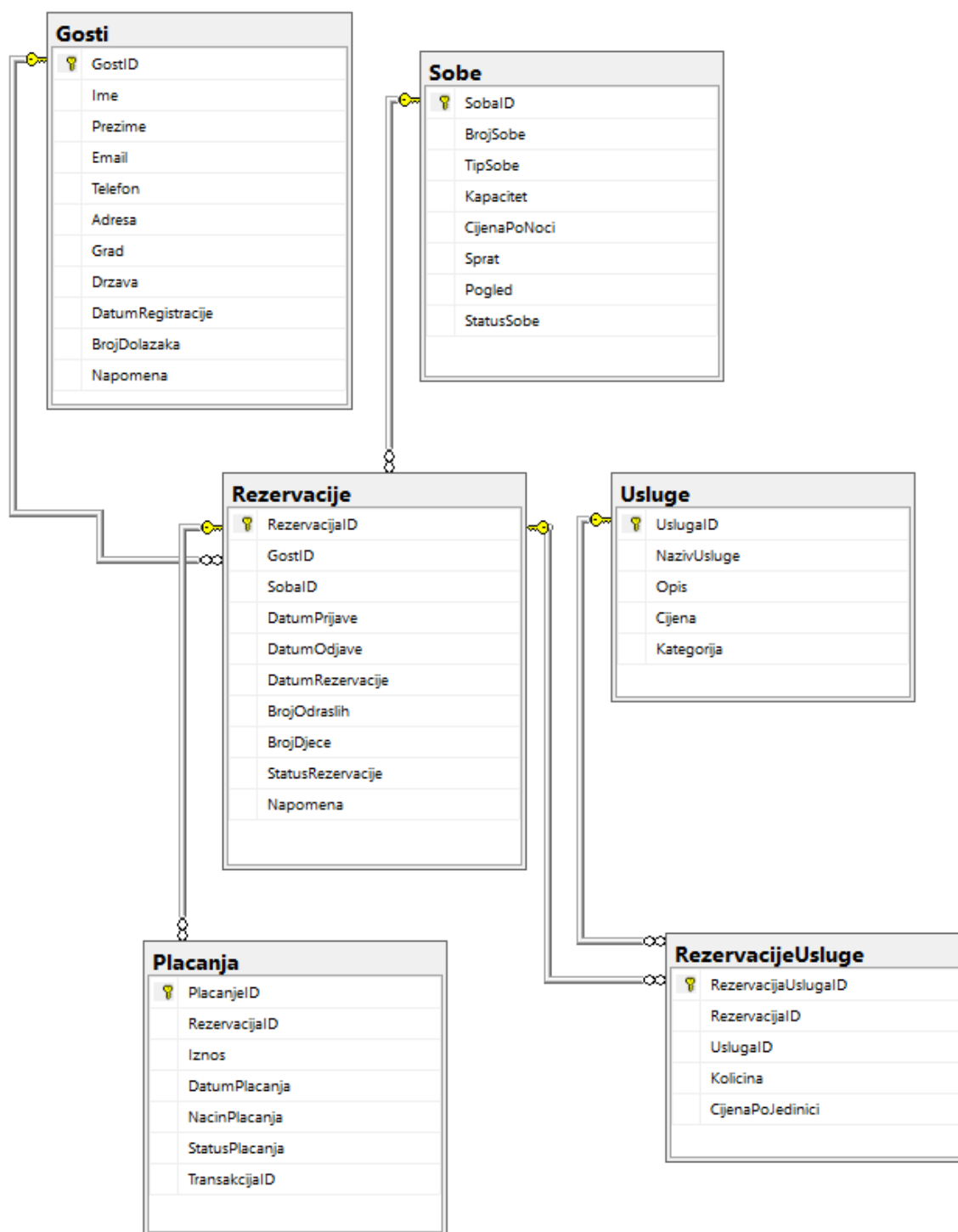
Entitet PLACANJA koristi se za evidentiranje svih finansijskih transakcija vezanih za rezervacije. U okviru njega se bilježe iznos, datum plaćanja, način plaćanja i status, što omogućava potpunu kontrolu nad finansijskim dijelom sistema.

15.2. Veze između entiteta

Između entiteta postoje jasno definisane relacije koje omogućavaju povezivanje podataka. Jedan gost može imati više rezervacija, dok svaka rezervacija pripada tačno jednom gostu. Slično tome, jedna soba može biti rezervisana više puta u različitim vremenskim periodima, ali svaka rezervacija se odnosi samo na jednu sobu.

Odnos između rezervacija i usluga je složeniji jer podrazumijeva vezu više-prema-više. To znači da jedna rezervacija može sadržavati više različitih usluga, dok se ista usluga može pojaviti u više rezervacija. Ovaj odnos je riješen pomoću dodatne tabele koja povezuje ova dva entiteta.

15.3. Grafički prikaz ER dijagrama



ZAKLJUČAK

U okviru ovog seminarskog rada uspješno je projektovana i implementirana baza podataka HotelRezervacije koja pokriva osnovne potrebe sistema za online rezervaciju hotelskih soba. Baza se sastoji od šest međusobno povezanih tabela koje omogućavaju evidenciju gostiju, soba, rezervacija, usluga i plaćanja.

Kroz pravilno definisane primarne i strane ključeve obezbijeđen je referencijalni integritet, dok su dodatna ograničenja osigurala tačnost unesenih podataka. Na taj način smanjena je mogućnost grešaka i obezbijeđena pouzdanost sistema.

Korištenjem indeksa unaprijeđene su performanse upita, posebno kod pretrage gostiju i rezervacija. Takođe, implementirane su procedure, funkcije i trigeri koji automatizuju ključne procese, poput kreiranja rezervacija, provjere dostupnosti i održavanja konzistentnosti podataka.

Dodatno, korišteni su pogledi za jednostavniji prikaz podataka, kao i backup mehanizmi koji omogućavaju sigurnost i oporavak baze u slučaju problema.

POPIS SLIKA

Slika 1 - Kreiranje baze podataka pomoću GUI-a	2
Slika 2 - Kreiranje baze podataka pomoću Query-a	2
Slika 3 - Aktiviranje baze podataka	3
Slika 4 - Postavljanje nivoa kompatibilnosti baze podataka na SQL Server 2014 (nivo 120)	3
Slika 5 - Selektovanje aktive baze putem GUI-a	3
Slika 6 - Prikaz tabele "Gosti"	4
Slika 7 - Query korišten za kreiranje tabele "Gosti"	5
Slika 8 - Query korišten za kreiranje tabele "Sobe"	6
Slika 9 - Query korišten za kreiranje tabele "Rezervacije"	7
Slika 10 - Query korišten za kreiranje tabele "Usluge"	8
Slika 11 - Query korišten za kreiranje tabele "RezervacijeUsluge"	8
Slika 12 - Query korišten za kreiranje tabele "Placanja"	9
Slika 13 - Ključna riječ za podešavanje primarnog ključa	10
Slika 14 - Dodatni način pronalazjenja primarnog ključa tabele pomoću GUI-a	10
Slika 15 - Primjer korištenja CHECK naredbe	11
Slika 16 - Prikaz nekih od indeksa u tabelama	14
Slika 17 - Postavljanje indeksa na kolone Prezime i Email u tabeli "Gosti"	15
Slika 18 - Postavljanje indeksa na datumske kolone u tabeli "Rezervacije"	15
Slika 19 - Postavljanje indeksa na kolonu StatusRezervacije u tabeli "Rezervacije"	15
Slika 20 - Postavljanje kompozitnog indeksa koji obuhvata više kolona istovremeno na kolonama "GostID" i "StatusRezervacije"	15
Slika 21 - Postavljanje kompozitnog indeksa koji obuhvata kolone TipSobe i CijenaPoNoci u tabeli "Sobe"	16
Slika 22 - Unos podataka u tabelu "Gosti"	17
Slika 23 - Unos podataka u tabelu "Sobe"	17
Slika 24 - Unos podataka u tabelu "Rezervacije"	18
Slika 25 - Unos podataka u tabelu "Usluge"	18
Slika 26 - Unos podataka u tabelu "RezervacijeUsluge"	19
Slika 27 - Unos podataka u tabelu "Placanja"	19
Slika 28 - Query korišten za kreiranje pogleda "vw_RezervacijeDetalji"	20
Slika 29 - Query korišten za kreiranje pogleda "vw_PrihodiPoSobama"	21
Slika 30 - Procedura kreirana za automatizaciju kreiranja novih rezervacija	22
Slika 31 - Procedura za automatsko iraćunavanje ukupne cijene rezervacije uzimajući u obzir smještaj i ostale usluge	23
Slika 32 - Procedura kreirana za potrebe izvještavanja o popunjenosti hotela u određenom mjesecu i godini	24
Slika 33 - Tabela koja čuva istoriju promjena cijena soba	25
Slika 34 - Query korišten za kreiranje trigera "trg_OtkaziRezervaciju"	26
Slika 35 - Query korišten za kreiranje trigera "trg_LogCijeneSoba"	26
Slika 36 - Query korišten za kreiranje trigera "trg_ProvjeraDuplihRezervacija"	27
Slika 37 - UDF za iraćunavanje popusta	28
Slika 38 - UDF za prikaz dostupnih soba za rezervaciju	29
Slika 39 - Tabela Rezervacije prije korištenja UPDATE naredbe	32
Slika 40 - Tabela Rezervacije nakon korištenja UPDATE naredbe na ID 92	32

Slika 41 - Query korišten za kreiranje nove tabele koja čuva istoriju cijena.....	34
Slika 42 - Primjer korištenja datumskih funkcija	35
Slika 43 - Korištenje String funkcija za manipulaciju teksta.....	36
Slika 44 - Primjer korištenja više agregatnih funkcija.....	36
Slika 45 - Primjer korištenja INNER JOIN-a	37
Slika 46 - Primjer korištenja LEFT JOIN-a.....	37
Slika 47 - Primjer korištenja RIGHT JOIN-a	38
Slika 48 - Primjer korištenja FULL OUTER JOIN-a	38
Slika 49 - Primjer korištenja CROSS JOIN-a.....	39
Slika 50 - Primjer korištenja SELF JOIN-a	39
Slika 51 - Primjer uspješnog backup-a cijelokupne baze podataka.....	40
Slika 52 - Primjer korištenja diferencijalnog backup-a	41
Slika 53 - Primjer korištenja Transaction Log Backup-a.....	41
Slika 54 - Primjer korištenja copy-only backup-a	41
Slika 55 - Primjer restauracije baze iz backup-a.....	42

RESURSI I DODATNI MATERIJALI

Radi bolje organizacije i transparentnosti rada, svi materijali korišteni tokom izrade ovog seminarskog projekta dostupni su na javnom repozitoriju na platformi GitHub.

U okviru repozitorija nalaze se SQL skripte za kreiranje baze podataka, definisanje tabela i relacija, kao i implementacija procedura, funkcija i trigera. Takođe su uključeni upiti korišteni za testiranje sistema, backup fajlovi i kompletan tekst seminarskog rada.

Korištenje repozitorija omogućava jednostavan pristup svim dijelovima projekta, kao i njihovu dalju nadogradnju i prilagođavanje. Na ovaj način obezbijedena je preglednost rješenja, ali i mogućnost verzionisanja i ponovne upotrebe koda u budućim projektima.

Materijali su dostupni na sljedećem linku:

<https://github.com/IvanPavlovic-web/apeiron/tree/main/Seminarski%20Radovi/Baze%20podataka>